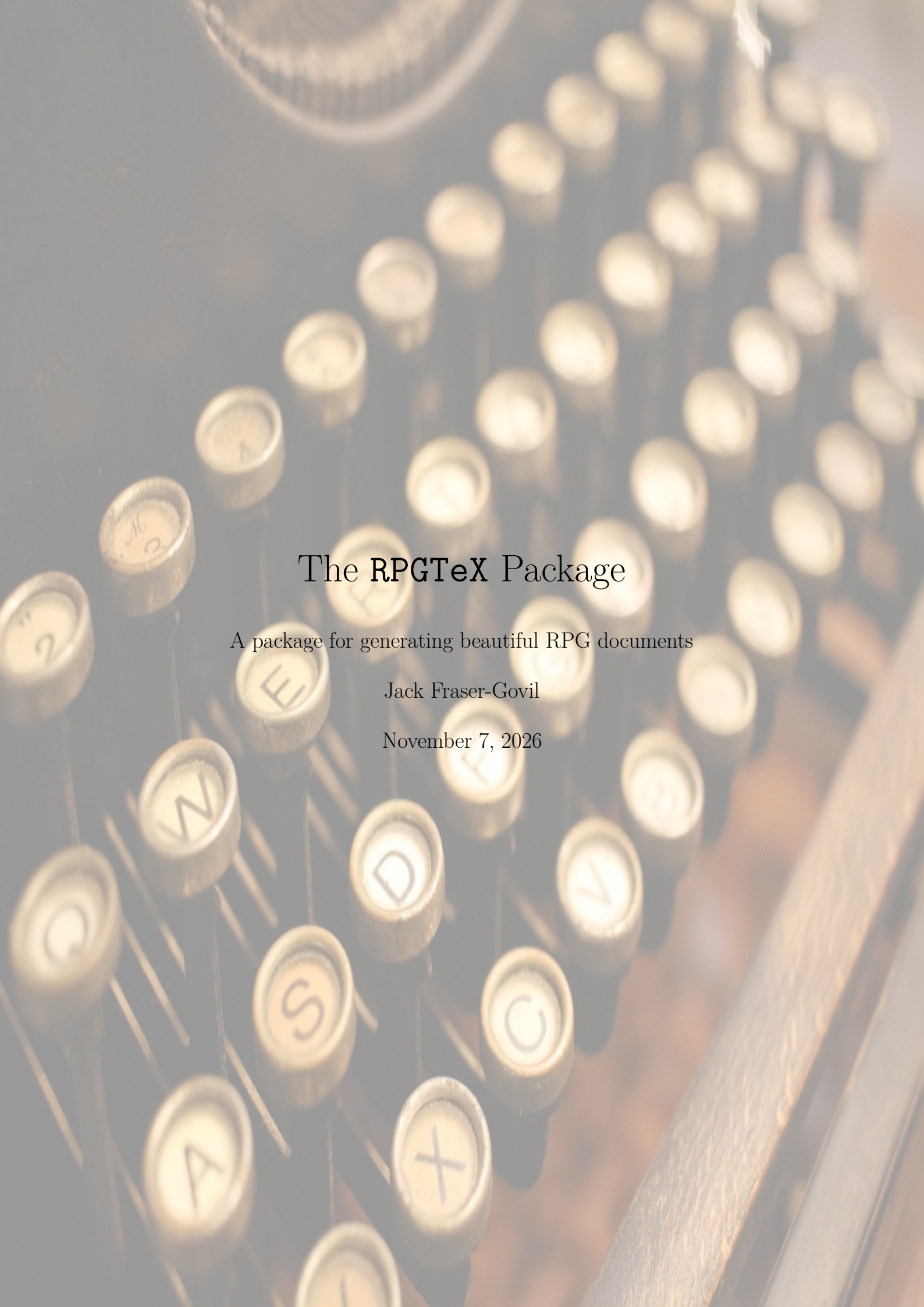


The RPGTeX Package

A package for generating beautiful RPG documents

Jack Fraser-Govil

November 7, 2026

Welcome to the **RPGTeX** *package.* This LaTeX package is designed to allow users to flexibly typeset documents associated with Role Playing Games (RPGs) such as *Dungeons & Dragons* – and many more besides. This package defines a central engine: `rpgcore` which defines a number of useful functions and classes, and a flexible set of `themes` which control how those commands are rendered in the final document.

Getting Started

This document is the full reference manual. For a quick example of how to get started with your first `RPGTeX` document, see the example documents, which demonstrate the basic features of this package in a readable fashion.

Attribution & Licence

This package would not have been possible without the team who developed its predecessor, the 'DND 5e LaTeX Template'. That code was released under an MIT licence, the text of which can be found in the LICENSE file. `RPGTeX` is released under the same licence.

Where third-party resources (fonts and images) have been included, they are credited in Appendix D and in the accompanying attribution files, and are distributed under the terms of their original licences.

AI Usage Declaration

The vast majority of this package was developed by hand by the human authors, though an AI assistant was used in the last period of development. All mistakes and bad decisions are organic in nature. A full breakdown of any AI-generated content is found in Appendix A

Contents

1: Installing RPGTeX
- Getting RPGTeX 1
- Compiling RPGTeX Documents 2

Part I: RPGTeX For Users

2: Package & Class Usage
- Choosing a Class 5
- Standalone Package 5
- rpgbook 6
- rpghandout 6
- rpgcard 6
- rpgdeck 6

3: Options & Configuration
- Package Options 8
- Colors 10
- Lengths 10
- Command Line Interface 11

4: Commands & Macros
- Themes & Colours 12
- Document Structure 12
- Text Utilities 15
- Full-Page Images 17
- Switches 17

5: Environments
- Callouts: RpgNarration, Sidebar & Tip 19
- RpgTables 20
- RpgAbstracts 21
- RpgFiligree 21
- RpgSecrets 22
- RpgMaps 23
- RpgCards 24
- Rule Environments 25

Part II: RPGTeX Themes

6: default Theme
- Appearance 28
- RpgItems 31
- RpgFeats 32
- RpgSpells & RpgStats 32

7: dnd Theme
- Appearance 33
- RpgItems 38
- RpgFeats 39
- RpgSpells 40
- RpgStats 42
- Customising the dnd Theme 52

8: scifi Theme
- Appearance 54
- Rule Environments 58

Part III: RPGTeX For Designers

9: Building a Theme
- Theme Files 61
- Conditional Code 62
- Key-Value Options 63

10: Fonts
- Font Commands 66
- Font Elements 67

11: Page Design
- Backgrounds & Footers 69
- Title & Part Pages 69

12: Styling Elements
- Box Formatters 71
- Other Formatters 71
- Table of Contents 74

13: Rule Environments
- Creating Rule Environments 75
- Properties 76
- Formats 76

Part IV: Appendices

Appendix A: AI Usage 80

Appendix B: The rpglatex Compiler 81

Appendix C: Dependencies 82
- Fonts 82

Appendix D: Image Credits 83

Appendix E: Index 84

Chapter 1: Installing RPGTeX

Getting RPGTeX

RPGTeX may be installed in several ways, described below. The package dependencies can be found in Appendix C (page 82). Once installed, RPGTeX attempts to locate its internal resources (fonts and images) automatically. If this fails, manual configuration may be required (see page 2).

Operating Systems

The commands below are written assuming a Linux or macOS terminal. Windows users should consult the section on page 1 for more information.

Direct Installation

The simplest approach is to install RPGTeX on the texmf path, where the compiler will find it automatically:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex" # make sure the directory exists
git clone https://github.com/DrFraserGovil/rpgtex.git "$(kpsewhich -var-value
    TEXMFHOME)/tex/latex/rpgtex"
```

This clones the repository into your personal LaTeX directory, which is typically ~/texmf on Linux, ~/Library/texmf on macOS, and C:\Users\<username>\texmf on Windows.

Indirect Installation

If you intend to modify RPGTeX – for example, by creating a new theme – it is more convenient to keep it somewhere easily accessible. First, clone the repository into a location of your choice:

bash

```
git clone https://github.com/DrFraserGovil/rpgtex.git ~/path/to/rpgtex
```

The package must then be made visible to the compiler, in one of two ways.

Using TEXINPUTS

The TEXINPUTS environment variable adds directories to the compiler's search path. The // includes all subdirectories of the given directory, and the trailing colon keeps the standard search locations:

bash

```
export TEXINPUTS=~/path/to/rpgtex//:
```

The exact command depends on your shell; add it to your shell's configuration file to make it permanent.

Using a Symbolic Link

Alternatively, a symbolic link from the texmf directory to the repository allows the compiler to behave exactly as if RPGTeX had been installed directly:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex"
ln -sf ~/path/to/rpgtex "$(kpsewhich -var-value TEXMFHOME)/tex/latex/rpgtex"
```

Windows

RPGTeX works on Windows, with either TeX Live or MiKTeX, with a few differences from the instructions above:

- Run the git commands using Git Bash for Windows.
- Use the direct installation. Symbolic links and `TEXINPUTS` behave differently on Windows, and are best avoided.
- If you use MiKTeX, it does not automatically detect new files in its directories. After installing, register the folder containing `RPGTeX` as a root directory in MiKTeX Console (if it is not already one), and then refresh the file name database (*Tasks* → *Refresh file name database*). TeX Live requires no extra steps.
- The `rpglatex` compiler (page 81) is only supported on Linux and macOS. On Windows, compile documents with `xelatex` or `lualatex` directly.

Overleaf

We provide a release bundle specifically configured to work out-of-the-box with Overleaf. To get this to work on Overleaf:

1. Navigate to the 'Releases' panel on GitHub, and select the most recent release.
2. Select the Overleaf bundle, `rpgtex-overleaf.zip` and download it.
3. Go to Overleaf.
4. Within the 'New Project' menu, select 'Existing project (.zip)', and upload the `rpgtex-overleaf.zip` file.

This bundle includes the `dnd`-theme example documents, to help get you started.

Watch your compile times

`RPGTeX` is a large package and as documents grow in size, compile times can the limits of the free tier of Overleaf. We generally recommend using local installs for large documents.

Compiling `RPGTeX` Documents

`RPGTeX` uses the `fontspec` package to load its fonts, and so must be compiled with `xelatex` or `lualatex` in almost all cases. An exception is if `RPGTeX` is loaded as a package without the font option (page 9):

```
xelatex main.tex # works
lualatex main.tex # works
pdflatex main.tex # fails when the font option is active
```

Path Configuration

In most cases, no further steps are required to compile a document, since `RPGTeX` finds its own installation path, and stores the value in the following variable:

`\RpgPackagePath` *(Variable)*

The path to the root directory of the `RPGTeX` installation. `RPGTeX` uses it to load resources, such as fonts, which TEX cannot find relative to the package itself. It is set by the first of the following which is available:

1. **The `rpglatex` compiler** (page 81), which defines it before the document is read.
2. **A configuration file**, `core/rpg-config.cfg`, within the installation.
3. **Automatically.** `RPGTeX` asks `kpsewhich` where `rpgtex.sty` is installed. This uses the compiler's restricted shell escape, which TeX Live enables by default, so no extra flags are needed.

If none of these succeeds, compilation stops with an error describing these options.

Failed Configuration

Most users will rely on the third option, the automatic configuration. However, this may fail if LATEX is (for example) running in a safe mode in which restricted writes and calls are not permitted. In such cases the user must either use the `rpglatex` compiler, run the Python script located at `scripts/configure-rpgtex`, or manually create the configuration file:

```
cd <rpgtex root directory>
echo '\edef\RpgPackagePath{'"$(pwd)"'}' > core/rpg-config.cfg
```

On some systems there can be a minor improvement in compilation speed by providing this file even in cases where the auto-config is valid. These will tend to be heavily sandboxed environments where external function calls come with a large penalty.

If `RPGTeX` is later moved, the file must be deleted or recreated.

PART I

RPGTeX For Users

Chapter 2: Package & Class Usage

RPGTeX may be loaded through one of its four classes, which set up the whole document, or as a package within a document of any other class. Most documents should use a class. The simplest complete RPGTeX document is:

```
\documentclass{rpgbook}
\begin{document}
  \chapter{The Adventure Begins}
  Your text here.
\end{document}
```

Options are given in the usual way, such as \documentclass[theme=dnd, columns=1]{rpgbook} (page 8).

Choosing a Class

Each class is suited to a different kind of document:

Class	Purpose
rpgbook (page 6)	Long documents, such as rulebooks and adventures, which are divided into parts and chapters.
rpgcard (page 6)	One or a few cards, such as an item or spell, cropped to size for viewing on screen.
rpgdeck (page 6)	Many cards, arranged on pages for printing.
rpghandout (page 6)	Shorter documents, such as player handouts, which are divided into sections.

What the Classes Provide

Compared with loading the package, every class:

- Activates the font, background and sections modules (page 8). Every class except rpgcard also activates the title module.
- Sets the page size (A4, except in the screen mode of rpgcard) and margins, and passes the size, columns, openright and oneside options to the underlying class.
- Applies RPGTeX's overall text layout: hyphenation is restricted, text is left-aligned unless the justified option is given, lists are spaced more compactly, and the table of contents is set in two columns unless the nomultitoc option is given.

Font Sizes

The size option sets the base font size, which must be one of the sizes accepted by the underlying class:

Classes	Sizes
rpgbook, rpghandout	8pt, 9pt, 10pt, 11pt, 12pt, 14pt, 17pt and 20pt
rpgcard, rpgdeck	10pt, 11pt and 12pt

Standalone Package

The package may be loaded into a document of any class:

```
\documentclass{<any-class>}
\usepackage[<options>]{rpgtex}
```

This loads the core commands and environments, and the theme (the default theme, unless the theme option is used). However, the modules which change the fonts, page backgrounds, section headings and title page are inactive unless they are enabled with the font, background, sections and title options, and none of the layout changes listed above (page 5) are made. This allows RPGTeX to be added to an existing document without changing its overall appearance.

rpgbook

The `rpgbook` class is designed for long documents, such as rulebooks, sourcebooks and adventures, which need to be organised into parts and chapters. It is the class used to typeset this manual.

It is based on the `extbook` class from the extsizes package, which behaves almost identically to the standard `book` class, but accepts a wider range of font sizes (page 5). All of the usual `book` commands are therefore available, including `\part`, `\chapter`, `\frontmatter`, `\mainmatter`, `\backmatter` and `\appendix`.

By default, `\maketitle` produces a full cover page (page 70). The `openright` option starts each chapter on a right-hand page, and the `oneside` option lays the document out for single-sided printing.

The page geometry is:

Element	Size
Left and right margins	0.65in
Top margin	0.4in
Bottom margin (from the text to the bottom of the page)	0.75in
Footer separation (from the text to the footer)	0.3in
Gap between columns	0.25in

rpghandout

The `rpghandout` class is designed for shorter documents, where the structure of a book is unnecessary: printouts for players, or short adventure modules.

It is based on the `extarticle` class from the extsizes package, which behaves almost identically to the standard `article` class, but accepts a wider range of font sizes (page 5). The highest sectioning level is therefore `\section`.

By default, `\maketitle` produces a compact title at the top of the first page, rather than a cover page (page 70). It is often followed by a short summary, using the `abstract` environment (page 21).

The page geometry is the same as that of `rpgbook` (page 6).

rpgcard

The `rpgcard` class is designed for typesetting one or a few cards (page 24), primarily for viewing on screen, or for collecting into a deck later (page 6). The rule environments are displayed as cards: the class turns on the switch of every cardable rule environment (page 75) which exists when the class is loaded. This includes the standard environments, such as `RpgItem` and `RpgSpell`, and any created by the theme given in the class options, but not those created later (in the preamble, or by a theme loaded within the document), whose switches must be turned on with `\RpgSwitch` (page 18). Other switches, such as `ShowSecrets`, are unaffected.

The class has two modes, chosen by the `bg` option (page 8):

Screen mode (`bg=full`, the default): the class is based on the `standalone` class, and each page is cropped to fit its contents exactly, giving a file which contains just the card.

Print mode (`bg=print` or `bg=none`): the class is based on the `article` class, and the cards are placed on A4 pages, with a margin of 5pt on each side. For printing large numbers of cards, `rpgdeck` is more suitable.

Cards cannot have a title page or a table of contents, and `\maketitle` raises an error. The `columns` option is ignored, since cards are always laid out in a single column.

In screen mode, the whole document is typeset as a single line, so everything should be placed within a card or rule environment: loose text and headings outside of a card cause errors. Several cards are placed side by side.

rpgdeck

The `rpgdeck` class is designed to collect a large number of cards (page 24) into a single document for printing, including cards which were written as separate `rpgcard` documents.

It is based on the standard `article` class, on A4 pages with a margin of 5pt on each side, and the cards are placed side by side, filling each page in turn. As in `rpgcard`, the rule environments are displayed as cards: the class turns

on the switch of every cardable rule environment which exists when the class is loaded, including the standard ones and any created by the theme given in the class options. Environments created later (in the preamble, or by a theme loaded within the document) must have their switches turned on with `\RpgSwitch` (page 18). The `columns` option is ignored, and no page numbers or footers are shown. Unlike `rpgcard`, a deck may have a title: `\maketitle` produces a compact title at the top of the first page (page 70).

With the default `bg=full`, the theme's paper background is drawn behind the cards across the whole page. When printing, `bg=print` avoids this.

Including rpgcard Documents

The `rpgdeck` class loads the `standalone` package with the `subpreambles=true` option. This allows a complete `rpgcard` document, with its own `\documentclass` and `document` environment, to be included in a deck with `\input`. For example, given the card:

```
% spell1.tex
\documentclass[theme=dnd]{rpgcard}
\begin{document}
  \begin{RpgSpell}{Fireball}
    ...
  \end{RpgSpell}
\end{document}
```

which may be compiled on its own (and, for example, sent to a player), several such cards may be collected into a deck:

```
% deck.tex
\documentclass[theme=dnd, bg=print]{rpgdeck}
\begin{document}
  \input{spell1}
  \input{spell2}
  ...
\end{document}
```

The preambles of the included cards are collected and added to the preamble of the deck, so any definitions they make are shared between all of the cards. Two cards which define the same command therefore conflict, and every card uses the last definition. To avoid this, definitions which differ between cards should be made within their `document` environments. The preambles are collected into an auxiliary file (`.sta`) during compilation, and are only read in on the next run, so a deck whose cards have preambles must be compiled twice. On the first run, anything which depends on them, such as an image found through `\graphicspath`, is silently missing.

The `\documentclass` line of an included card is ignored, along with its options. The cards are therefore typeset with the deck's theme and options, which may differ from the ones used when the card was compiled on its own; `\RpgSetTheme` (page 12) may be used within the deck to change the theme.

Chapter 3: Options & Configuration

This chapter describes the ways in which the behaviour of RPGTeX may be configured without writing any new code: the options which may be passed to the package or class, the named colours and lengths used by its elements, and the command line interface for changing these options at compile time.

Package Options

Whether RPGTeX is loaded directly as a package or indirectly through one of its classes, options may be passed to it which change the behaviour of the resulting document:

```
\documentclass[<options>]{rpgbook}
\usepackage[<options>]{rpgtex}
```

Each option takes one of two forms:

Key-value options: The name of the option is followed by an equals sign and a value:

```
\usepackage[bg = print, theme = dnd]{rpgtex}
```

Flags: Only the name of the option is provided. Its presence enables the associated behaviour:

```
\usepackage[justified]{rpgtex}
```

The available options are:

`background` <flag>

If present, activates the 'paper background' modules, which control the paper colour and background images, the footer decorations, and the spacing around floats.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`bg` = full / print / none

Controls the behaviour of the background and footer decorations. The value must be one of the three listed above, else an error is thrown. The default value is `full`.

The most visible effect of this option is to control whether the 'background paper' set by `\RpgThemePaperImage` (page 69) and `\RpgThemePaperColor` (page 69), and the footer decorations set by `\RpgThemeFooterDecoration` (page 69), appear in the document:

Value	Paper	Footer Decorations
`full`	✓	✓
`print`	✗	✓
`none`	✗	✗

This option also changes the behaviour of the rpgcard class (page 6), and other environments may similarly alter their colours or layouts in response to its value.

`columns` = 1 / 2

Places the document into either `onecolumn` or `twocolumn` mode. The default value is `2`.

More advanced column layouts require the user to manually use the `multicols` environment.

This option only has an effect when using one of the RPGTeX classes.

`font` <flag>

If present, activates the font modules, applying the package fonts wherever possible.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`justified` <flag>

If present, sets the text to be justified throughout the document. By default, text is left-aligned ('ragged right').

`noemph` <flag>

If present, the default behaviour of `\emph` is left unchanged. By default, `\emph` is redefined to be an alias of `\RpgKey` (page 16).

`nomultitoc` <flag>

If present, disables the multi-column table of contents.

`oneside` <flag>

If present, places the document into one-sided mode. This disables alternating footers, but allows for consistent asymmetric margins (as in this document, where the left margin is enlarged to accommodate macro names).

This option only has an effect when using one of the `RPGTeX` classes.

`openright` <flag>

If present, places the document into right-opening mode, such that chapters always begin on a right-hand page. By default, chapters may begin on any page (see the documentation of the `book` class for more details).

This option only has an effect when using one of the `RPGTeX` classes.

`sections` <flag>

If present, activates the sections modules, including the part-page overloads and any theme behaviour which modifies the section commands.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`size` = <font-size>pt

Sets the base font size of the document, if permitted by the parent class. The default value is `10pt`.

This option only has an effect when using one of the `RPGTeX` classes.

The allowed values depend on the class being used (page 5).

`theme` = <theme name>

The initial value passed to `\RpgSetTheme` (page 12) once package initialisation is complete.

The default value is `default`, which activates the Default Theme (page 28).

`themepath` = </path/to/themes/>

Calls `\RpgSetThemePath` (page 12) before the theme is loaded, allowing custom themes to be used.

The default value is `\RpgPackagePath/themes`, which TEX locates within the `RPGTeX` installation. A theme called `name` is then expected to be stored in `<themepath>/name/name.cfg`.

`title` <flag>

If present, activates the titling module and any theme presets which alter the behaviour of `\maketitle`.

This flag is set by default when loading a class, but must be passed explicitly to enable this module when loading the package directly.

Colors

RPGTeX defines a number of named colors which are used by its various elements. The default values listed below may be overridden by the active theme.

A Note on Spelling

Although this document otherwise uses British spelling, this section follows the American spelling of 'color' used throughout LaTeX and xcolor. The color names below must be written exactly as given, including their spelling and capitalization.

An exception is in Gray/Grey, as I personally cannot remember which is the 'right' way to spell it, so I have aliased most colours to use both spellings.

Co-varying Colors

The following colors are coupled together, and by default all take the same value:

Color	Usage	Default
SidebarColor	The background color of the RpgSidebar (page 19) environment.	ThemeColor
TableColor	The background color of alternate rows in an RpgTable (page 20).	ThemeColor
ThemeColor	A general-purpose color from which the other co-varying colors inherit.	white!60!black
TipColor	The background color of the RpgTip (page 19) environment.	ThemeColor

Calling \RpgSetThemeColor (page 12) assigns a new value to ThemeColor and all three co-varying colors simultaneously:

```
\RpgSetThemeColor{blue!20!white}
```

Independent Colors

The remaining colors are set individually:

Color	Usage	Default
ContourInnerColor	The default color of the inner text produced by \RpgContour (page 15).	black
ContourOuterColor	The default color of the outer contour drawn around text by \RpgContour (page 15).	white!50!black
FiligreeBackColor	The background color of the RpgFiligree (page 21) environment.	white
FiligreeColor	The color used to draw the filigree frame applied by the rpgfiligree style.	black
NarrationColor	The background color of the RpgNarration (page 19) environment.	F7F2E5 (HTML)
PaperColor	The background color of each page, applied only when the paper background is enabled (see the bg option, page 8).	white
SecretColor	The default frame color of the RpgSecret (page 22) environment.	black!30!white

Any of these colors, including the co-varying colors, may be modified individually using the standard xcolor interface:

```
\definecolor{NarrationColor}{HTML}{FFFFFF}
\colorlet{SidebarColor}{red!10!white}
```

Modifying a co-varying color in this way changes only that color, leaving the others unaffected.

Lengths

A small number of lengths used by RPGTeX may be changed directly, without any special interface. As with the colors, the active theme may override their default values.

\RpgDropCapSeparation *(Variable)*

The horizontal space between the enlarged letter created by `\RpgDropCap` (page 16) and the text which follows it. It is a standard LaTeX length, and is changed with `\setlength`:

```
\setlength{\RpgDropCapSeparation}{0.5em}
```

The default value is the width of a single space.

`\RpgFiligreeSize` *(Variable)*

The size of the decorative elements of the filigree frame (page 21), which the theme uses to scale its design. Unlike a standard length, it is stored as a macro, and so is changed with `\renewcommand`:

```
\renewcommand{\RpgFiligreeSize}{8pt}
```

The default value is `5pt`.

Command Line Interface

LaTeX does not natively provide a 'command line interface' through which a document may be modified at compile time: any change must be made within the source file itself. It is, however, often convenient to compile the same document in several configurations without editing it – most notably to produce both a standard version and a printer-friendly version (using `bg = print`, which removes the paper background to reduce ink usage).

To support this, `RPGTeX` checks for a pseudo command-line variable when processing its options. Because TeX treats the text passed to it on the command line as the beginning of its input stream, this variable can be defined before the document itself is read.

`\RpgCMD` *(Variable)*

If defined, holds a comma-separated list of package options (page 8), which are applied after all other options have been processed. The options are applied in the following order, with later values overriding earlier ones:

1. The defaults set by the class (if one is being used).
2. The options passed in the document, via `\documentclass` or `\usepackage`.
3. The contents of `\RpgCMD`.

Since the options are processed before they are passed to the parent class, options which affect the class (such as `columns` or `size`) may also be set in this way.

`\RpgCMD` is defined by passing a short piece of TeX code to the compiler before the document is input:

```
xelatex "\def\RpgCMD{<rpg-options>} \input{<document>}"
```

This compiles `<document>` as though `<rpg-options>` had been passed to the class or package. For example, the following produces a printer-friendly, single-column version of a document without modifying the source:

```
xelatex "\def\RpgCMD{bg=print, columns=1} \input{document.tex}"
```

The `rpglatex` compiler (page 81) performs this insertion automatically: the `-p` (`--print`) flag sets `bg=print`, allowing the user to switch between the standard and print versions of a document with a single flag:

```
rpglatex document.tex -p
```

Chapter 4: Commands & Macros

Themes & Colours

`\RpgSetTheme { name }`

Loads the theme called `name`, replacing the appearance of any previously loaded theme. The theme is first searched for at `<themepath>/<name>/<name>.cfg`; if no such file exists, `name` is instead treated as a direct path to a theme file. An error is raised if neither file exists.

The initial theme is set by the `theme` option (page 8). `\RpgSetTheme` may be called at any point in the document, but since its effects are immediate, it is best called at the start of a new page.

`\RpgSetThemeColor { colour }`

Sets `ThemeColor` and all of the co-varying colours to `colour` (page 10).

`\RpgSetThemePath { path }`

Sets the directory searched by `\RpgSetTheme`, allowing custom themes to be stored outside of the `RPGTeX` installation. The path is not checked until a theme is loaded. The initial value is set by the `themepath` option (page 9).

Document Structure

`RPGTeX` extends the standard LaTeX commands for structuring a document. It provides title and part pages designed by the active theme, records the names of the current headings so that they may be reused elsewhere, and allows 'fake' headings to be added to the table of contents.

The Title & Sections Options

Much of the behaviour described in this section is provided by redefining standard LaTeX commands, and is controlled by two package options (page 8). Both are active by default when using one of the `RPGTeX` classes; otherwise, they must be passed to the package explicitly.

`title` Redefines `\maketitle` to produce the title designed by the active theme (equivalent to calling `\RpgMakeTitle`). Otherwise, `\maketitle` retains its original LaTeX behaviour.

`sections` Activates ***sections mode***, which:

- Redefines `\chaptermark`, `\sectionmark`, `\subsectionmark` and `\subsubsectionmark` so that, in addition to their usual behaviour, they store the name they are given (page 14).
- Sets `secnumdepth` to `0`, so that sections and their subdivisions are unnumbered.
- Replaces `\part` with a version which records the part name and draws a full-page part page using the active theme. The starred form, `\part*`, records the name but otherwise falls back to the original LaTeX command.

Themes may additionally alter the formatting of the sectioning commands when sections mode is active.

All of the other commands in this section, including `\RpgMakeTitle`, are available regardless of these options, although the stored heading names are only updated in sections mode.

Title Pages

The title page draws on the usual `\title` and `\author` commands, along with the following additions.

`\cover { path/to/image }`

Sets an image to be used as the full-page background of the cover page created by `\maketitle` in `cover` mode (see below). The theme's title design is drawn over the top of the image.

The default value is empty, in which case the title design is drawn on a plain page.

`\maketitle {}`

`\RpgMakeTitle`

Creates the title of the document using the design provided by the theme (only when the `title` option is active; see above). There are two title modes:

`cover` A full-page cover, drawn over the `\cover` image (if one has been set). This is the default for the `rpgbook` class.

`header` A title heading placed at the top of the first page of text.

The mode is selected with `\RpgThemeTitleMode{ cover / header }`. If the theme does not define a title for the active mode, the original LaTeX `\maketitle` is used instead.

`\RpgMakeTitle` is equivalent to `\maketitle`, but remains available even if the `title` option is not active, or if another package redefines `\maketitle`.

`\RpgCoverTransparency { value }`

Fades the `\cover` image towards white, which can help to keep the title legible over a busy image. The `value` is the opacity of a white layer placed over the image, between `0` (the image is unaltered) and `1` (the image is completely hidden). The default value is `0`.

`\RpgGetAuthor` *(Variable)*

`\RpgGetAuthor*`

The author set by `\author`, formatted with the `\RpgFontSubTitle` (page 67) font element. The starred form gives the author as plain text.

`\RpgGetSubtitle` *(Variable)*

`\RpgGetSubtitle*`

The subtitle set by `\subtitle`, formatted with the `\RpgFontSubTitle` (page 67) font element. The starred form gives the subtitle as plain text.

`\RpgGetTitle` *(Variable)*

`\RpgGetTitle*`

The title set by `\title`, formatted with the `\RpgFontTitle` (page 67) font element (page 66). The starred form gives the title as plain text, without applying the font.

`\subtitle { text }`

Sets the subtitle of the document. As with `\title` and `\author`, this has no visible effect by itself: the subtitle is displayed by `\maketitle` if the theme includes it. The default value is empty.

Part Pages

`\part (*)[<image>]{ name }`

In sections mode (page 12), `\part` is redefined to create a full-page part page using the design provided by the theme. Its behaviour depends on the form used:

`\part{name}` Adds the part to the table of contents and creates a part page on a plain background, or over the default part image if one has been set with `\RpgDefaultPartImage`.

`\part[image]{name}` As above, but uses `image` as the full-page background.

`\part*{name}` Uses the original `\part` command of the underlying class. Any image argument is ignored.

In all cases, `name` is recorded in `\RpgPartName` (page 14).

`\RpgDefaultPartImage { path/to/image }`

Sets an image to be used as the full-page background of every part page which is not given its own image. By default, no image is used.

`\RpgPartTransparency { value }`

Fades all subsequent images set by `\part` image towards white, which can help to keep the text legible over a busy image. The `value` is the opacity of a white layer placed over the image, between `0` (the image is unaltered) and `1` (the image is completely hidden). The default value is `0`.

Section Names

Many RPG documents display the name of the current chapter or section outside of the heading itself – in a running header, a footer, or a decorative page border, for example. To support this, `RPGTeX` records the names of the current part, chapter, section, subsection and subsubsection, so that they may be reused anywhere in the document.

These names are only recorded in sections mode (page 12).

Each of the following variables is initially blank, and holds its value until the next command of the same level is called: beginning a new chapter does not, for example, reset `\RpgSectionName`.

`\RpgChapterName` *(Variable)*

The name given to the last `\chapter` command, or set by calling `\chaptermark`. Starred chapters (`\chapter*`) do not update this value.

`\RpgPartName` *(Variable)*

The name given to the last `\part` or `\part*` command.

`\RpgSectionName` *(Variable)*

The name given to the last `\section` command, or set by calling `\sectionmark`. Starred sections (`\section*`) do not update this value.

`\RpgSubsectionName` *(Variable)*

The name given to the last `\subsection` command, or set by calling `\subsectionmark`. Starred subsections (`\subsection*`) do not update this value.

`\RpgSubsubsectionName` *(Variable)*

The name given to the last `\subsubsection` command, or set by calling `\subsubsectionmark`. Starred subsubsections (`\subsubsection*`) do not update this value.

`\RpgGetName { part / chapter / section / subsection / subsubsection }`

Returns the name stored for the given level (equivalent to the corresponding variable above). Any other input results in an error.

Since these names (other than that of the part) are taken from the `mark` commands, they may be set manually, without creating a new heading, by calling the relevant `mark` command directly.

```
We are in Chapter
  ‘\RpgGetName{chapter}’, section
  ‘\RpgGetName{section}’.

\subsubsection{An example subsubsection}

   Now I am in
     ‘\RpgGetName{subsubsection}’.
```

We are in Chapter ‘Commands & Macros’, section ‘Document Structure’.

An example subsubsection

Now I am in ‘An example subsubsection’.

Fake Sections

It can be helpful to be able to integrate common RPG elements (such as full page maps or images) into the document structure. The following command provides a simple interface for spoofing in a section (or part, chapter etc.) into the table of contents, without actually invoking a sectioning command:

`\RpgFake { level }{ name }`

Mimics the creation of a new heading at the given `level` without printing a heading in the document. Specifically, it:

- Adds an entry called `name` to the table of contents at the given level, with a hyperlink anchor at the current position.
- Updates the stored name for that level, as if a real heading had been created. For `chapter` to `subsubsection` this is done by calling the corresponding `mark` command, so any running headers are also updated.

The `level` must be one of `part`, `chapter`, `section`, `subsection` or `subsubsection`. Any other value results in an error.

```
\subsubsection{Real Subsection}
I believe I am in
  ‘\RpgGetName{subsubsection}’.

\RpgFake{subsubsection}{Fake Subsection}
Now I believe I am in
  ‘\RpgGetName{subsubsection}’
```

Real Subsection

I believe I am in ‘Real Subsection’.
Now I believe I am in ‘Fake Subsection’

Text Utilities

`RPGTeX` provides a number of commands for applying decorative styles to text and transforming it in ways useful for writing in an RPG context.

Any fonts or styles they apply are those assigned by the theme (page 66).

\RpgContour [<options>]{ text }

Typesets `text` with a coloured outline (contour), using the `contour` package. The following options are available:

Option	Effect	Default
`inner`	The colour of the text itself.	`ContourInnerColor` (page 10)
`outer`	The colour of the outline.	`ContourOuterColor`
`style`	Formatting commands (such as `\bfseries` or `\Huge`) applied to the text.	(empty)

Contours cannot automatically linebreak, instead the text is split into lines at each `\\`, and each word is contoured individually so that spacing is preserved. As a consequence, formatting commands placed within `text` may only affect the first word: all formatting should therefore be passed through the `style` key.

```
\RpgContour[inner=red,
  outer=black]{Contoured text}

\RpgContour[style=\Large\bfseries]{Styled
  text \\ over two lines}
```

Contoured text
Styled text
over two lines

\RpgContourIfDifferent [<options>]{ text }

Functions similarly to `\RpgContour`, but first checks the `inner` and `outer` colours:

If Different The function is identical to `\RpgContour`

If the Same The function renders the text without a contour (but with the relevant styling and colour)

This prevents excess contouring which can make text look blocky. However, since contoured text has strict non-linebreaking behavour, this means that changing the colours can have a structural difference on the text.

\RpgDice { expression }

Parses a dice expression of the form `<count>d<sides><modifier>` and typesets it in the format defined by the theme. The expression must satisfy the following rules:

- It contains a dice symbol (`d` or `D`), immediately followed by the number of sides.
- The dice symbol may be preceded by the number of dice. If it is omitted, the theme determines how the expression is displayed.
- Anything following the number of sides is treated as the modifier, which may be any numerical expression (such as `+3*2`). It is evaluated to a single number before being displayed.

Leading and trailing spaces are ignored. If the expression does not match this form, it is printed unchanged.

```
\RpgDice{1d6-2}

\RpgDice{2D6 + 3*2^2}

\RpgDice{d8}

\RpgDice{not a dice roll}
```

1d6-2
2d6+12
d8
not a dice roll

\RpgDropCap [<options>]{ letter }{ text }

Begins a paragraph with a large decorative initial (a 'drop cap'), using the `lettrine` package. The `letter` is typeset in the `\RpgFontDropCap` (page 68) font element, and `text` – intended to be the remainder of the first line – in the `\RpgFontDropCapAfter` (page 68) font element.

The drop cap spans a number of lines set by the theme (four, by default), and is separated from the following text by the length `\RpgDropCapSeparation` (page 10). The optional argument accepts any `lettrine` options, which override these defaults.

```
\RpgDropCap{T}{he example: this text
  runs over the first line, and then
  revert back to the normal font. It
  almost works! However, because it's
  wrapped in a text box, it goes
  slightly over the edges.}
```

The example: this text runs over the first line, and then revert back to the normal font. It almost works! However, because it's wrapped in a text box, it goes slightly over the edges.

\RpgKey { text }
\emph

Typesets `text` in the `\RpgFontEmphasis` (page 67) font element, giving a consistent appearance to keywords (such as conditions or game terms).

When the `font` option is active, and unless the `noemph` option is passed (page 8), `\emph` is redefined to be an alias of `\RpgKey`. This replaces its usual 'context-aware' behaviour (italics within upright text, and vice versa) with the keyword style.

```
\emph{text}

\RpgKey{more text}
```

text
more text

\RpgOrdinal [<format>]{ number }

Typesets an integer followed by its English ordinal suffix (`st`, `nd`, `rd` or `th`), correctly handling `11th`, `12th` and `13th`. The optional `format` is a command applied to the suffix only, such as `\textbf`.

```
\RpgOrdinal{1}, \RpgOrdinal{12},
  \RpgOrdinal{23}

\RpgOrdinal[\textbf]{133}
```

1st, 12th, 23rd
133**rd**

\RpgPage [<style>]{ label }

Typesets 'page' followed by the page number of `label`, optionally enclosed in brackets. The `style` may be one of `t` (no brackets; the default), `p` (parentheses), `b` (square brackets) or `c` (curly braces). Any other value encloses the reference in question marks, to make the error easy to spot.

```
\RpgPage{S:PackageOptions}
\RpgPage[p]{S:PackageOptions}
\RpgPage[b]{S:PackageOptions}
\RpgPage[c]{S:PackageOptions}
```

page 8
(page 8)
[page 8]
{page 8}

`\RpgPlural` `(*)[<plural>]{ count }{ word }`

Typesets `count` followed by `word`, adding an 's' to the word unless `count` is `1`. Irregular plurals may be given in the optional argument, which replaces the word whenever `count` is not `1`. The starred form omits the count, printing only the word.

```
\RpgPlural{1}{hat}, \RpgPlural{2}{hat}

\RpgPlural[octopodes]{1}{octopus},
  \RpgPlural[octopodes]{3}{octopus}

\RpgPlural*{3}{hat}
```

1 hat, 2 hats
1 octopus, 3 octopodes
hats

`\RpgSuperOrdinal` `{ number }`

Equivalent to `\RpgOrdinal[\textsuperscript]{ number }` (page 16), automatically applying a superscript style command.

```
\RpgSuperOrdinal{42}
```

42nd

Full-Page Images

We use the `eso-pic` library for formatting whole-page images and placing content at specific positions on the page.

`\RpgPut` `[<width>][<format>](<anchor>){ x }{ y }{ content }`

Places `content` at a fixed position on the page, measured from the bottom-left corner. It may only be used within a page overlay, such as the first argument of `\RpgWholePageImage`.

The content is set in a paragraph of the given `width` (default: `\linewidth`), with `format` applied at the start (default: `\centering`).

The `anchor` determines which point of the content is placed at (`x`, `y`): `c` (centre; the default), `t`, `b`, `l`, `r`, or a combination such as `tl`.

`\RpgWholePageImage` `[<overlay>]{ path/to/image }[<fade>]`

Places an image so that it fills the whole of a new page, clearing the page on either side. This is useful for maps, handouts and illustrations.

The optional `overlay` is drawn on top of the image, and may contain any number of `\RpgPut` commands. The final optional argument (given in square brackets, after the path) fades the image towards white, as described for `\RpgCoverTransparency` (page 12).

Switches

Switches are named true/false values which can be used to change the content of a document without editing its text. For example, `RPGTeX` uses switches to decide whether secrets are shown, and whether rule environments are displayed as cards. All switches are global: changing a switch affects the whole of the remaining document.

Switches are designed to act as boolean counterparts to traditional TEX counters, and so have a similar interface.

The primary use-case for switches in `RPGTeX` is for the 'Cardable Environments' (page 75). , allowing the same document to typeset as both inline text, and as a 'playing card module' depending on the value of the associated switch.

`\RpgIfSwitch { name }{ if-on }{ if-off }`

Typesets `if-on` if the switch is on, and `if-off` otherwise.

`\RpgNewSwitch { name }[<value>]`

Creates a new switch called `name`, with an initial `value` (default: `false`). An error is raised if the switch already exists.

`\RpgSwitch { name }{ value }`

Sets an existing switch to the chosen value. Accepted input strings are:

Boolean State	Input Strings
True	on, On, true, True, 1
False	off, Off, false, False, 0

Any other value raises an error, as does a switch which does not exist.

`\RpgToggle { name }`

Turns an existing switch on if it is off, and off if it is on.

```
\RpgNewSwitch{HardMode}

The dragon has
  \RpgIfSwitch{HardMode}{500}{200} HP.

\RpgToggle{HardMode}

The dragon has
  \RpgIfSwitch{HardMode}{500}{200} HP.
```

The dragon has 200 HP.
The dragon has 500 HP.

Chapter 5: Environments

RPGTeX provides a number of environments for the elements which appear throughout RPG books: boxed text, tables, map keys, secrets and playing cards. Their appearance (fonts, colours and decoration) is set by the active theme, but may be adjusted through their options.

Many of these environments are built on the tcolorbox package. Unless stated otherwise, any option which is not recognised by an environment is passed on to tcolorbox, so all of its usual options (such as colframe or breakable) may also be used.

Callouts: RpgNarration, Sidebar & Tip

Callouts are boxed blocks of text which stand apart from the main text, the precise appearance determined by the theme. There are three types of callout, intended for different purposes:

RpgNarration	Text to be read aloud, such as the description of a scene at the start of an adventure.
RpgSidebar	Important information, such as key rules or summaries, which readers *should* pay attention to.
RpgTip	Helpful additions, such as tips and trivia, which are not essential but are too large for a footnote.

Each callout accepts the following options, in addition to the usual tcolorbox options:

Option	Effect	Default
color	The background colour of the box and its title, setting both colback and colbacktitle. An explicit colback or colbacktitle option takes priority over this value.	NarrationColor, SidebarColor or TipColor (page 10)
filigree	If present, decorates the box with the theme's filigree frame (page 21).	off

\begin{ RpgNarration } [<options>]

A box for text to be read aloud. It has no title, and its text is typeset in the \RpgFontNarration (page 68) font element on a NarrationColor background.

```
\begin{RpgNarration}[color=blue!30!white]
   This is text that you would read out
     loud to players, describing a
     scene. It will always be blue,
     even if the theme says otherwise
     -- because the optional argument
     takes priority.
\end{RpgNarration}
```

This is text that you would read out loud to players, describing a scene. It will always be blue, even if the theme says otherwise – because the optional argument takes priority.

\begin{ RpgSidebar } [<options>]{ title }

A box for important information, with a title. The title and body are typeset in the \RpgFontSidebarTitle (page 68) and \RpgFontSidebarBody (page 68) font elements, and the background is SidebarColor.

```
\begin{RpgSidebar}{A Sidebar}
   This is an important block of text,
     that you should pay attention to.
\end{RpgSidebar}
```

A Sidebar
This is an important block of text, that you should pay attention to.

\begin{ RpgTip } [<options>]{ title }

A box for helpful additions, with a title. The title and body are typeset in the \RpgFontTipTitle (page 68) and \RpgFontTipBody (page 68) font elements, and the background is TipColor.

```
\begin{RpgTip}{A Tip}
   This is some helpful - but not vital
     - text.
\end{RpgTip}
```

A Tip

This is some helpful - but not vital - text.

RpgTables

`\RpgRowStyle { format }`

Applies `format` to every cell of the current row of an `RpgTable`. It must be placed at the start of the row. This is the mechanism used to style the header row, and may be used to highlight any other row in the same way.

`\begin{ RpgTable } [<options>]{ columns }`

Typesets a table in the style of the theme. By default this is a `tabularx` table which fills the width of the line, so the usual column types (`l`, `c`, `r`, `p{<width>}`, …) may be used alongside the stretching `X` column. `RPGTeX` also provides a centred stretching column, `Y`.

By default, the table:

- Typesets its first row as a header, in the `\RpgFontTableHeader` (page 68) font element, and the remaining rows in `\RpgFontTableBody` (page 68).
- Colours alternate rows in `TableColor` (page 10), beginning with the second row.

The following options are available:

Option	Effect	Default
`breakable`	If present, uses `xltabular`, allowing the table to break across pages. This is only possible in one-column mode.	off
`color`	The colour used for the coloured rows.	`TableColor` (page 10)
`color-both`	If present, colours every row.	off
`color-odd`	If present, colours the odd rows (including the header) instead of the even rows.	off
`filigree`	If present, wraps the table in an `RpgFiligree` (page 21).	off
`filigree-hgap`	The horizontal padding between the filigree and the table.	`1pt`
`filigree-opacity`	The opacity of the filigree's background.	`1`
`headskip`	The vertical space below the title.	`5pt plus 2pt minus 2pt`
`intro`	Introductory text, typeset between the title and the table.	(empty)
`introskip`	The vertical space below the introduction.	`5pt plus 2pt minus 2pt`
`noheader`	If present, the first row is typeset in the same style as the rest of the table.	off
`simple`	If present, uses a plain `tabular`, whose width is determined by its contents. The `width`, `breakable` and `noheader` options are then ignored, and the first row is not styled as a header.	off
`title`	A title, typeset above the table in the `\RpgFontTableTitle` (page 68) font element.	(empty)
`vskip`	The vertical space above and below the table.	`9pt plus 3pt minus 3pt`
`width`	The width of the table.	`\linewidth`

```
\begin{RpgTable}[width=0.85\linewidth,
  color=green!30!white]{lX}
    Header 1 & Header 2
    \\
    Text & Text fills up to 85\% of the
      line width then breaks
    \\
    Alternating & This row is transparent
    \\
    Colour & but this one is green
\end{RpgTable}

%suppress header formatting, add title
  and filigree:
\begin{RpgTable}[title={Test
  Table},noheader, filigree]{XlX}
    Plain & Header & Text
    \\
    Now there's no & difference &
      between the header and the main
    \\
    body
\end{RpgTable}
```

Header 1	Header 2
Text	Text fills up to 85% of the line width then breaks
Alternating	This row is transparent
Colour	but this one is green

Test Table

Plain	Header	Text
Now there's no	difference	between the header and the main
body		

RpgAbstracts

`\begin{ RpgAbstract }` `[<title>][<width>][<gap>]`

`abstract`

Typesets a short, centred summary block, such as the introduction to an adventure. The `title` (default: 'Abstract') is typeset in the `\RpgFontAbstractTitle` (page 68) font element, and the body in `\RpgFontAbstractBody` (page 68). The block has the given `width` (default: `0.7\linewidth`), with a vertical `gap` above and below it (default: `15pt`).

In two-column mode, the block spans both columns. The standard `abstract` environment is redefined to be identical to `RpgAbstract`.

`\begin{ summary }` `[<title>][<width>][<gap>]`

Identical to `RpgAbstract`, but with the default title 'Summary'.

RpgFiligree

Filigree is a decorative frame, drawn around the outside of a box in place of its usual border. Its design is provided by the theme, and it is drawn in `FiligreeColor` (page 10).

`\begin{ RpgFiligree }` `[<options>]`

A plain box decorated with the filigree frame, with a `FiligreeBackColor` background. It is equivalent to an untitled `RpgTip` (page 19) with the `filigree` option, and accepts the same options.

The filigree may also be added to any `tcolorbox` by including the `rpgfiligree` style in its options, or to a callout or table with the `filigree` option.

```
%% As environment
\begin{RpgFiligree}
    A basic filigree-wrapped tcbox.
\end{RpgFiligree}


%% As option
\begin{RpgSidebar}[filigree]{Fancy
  Sidebar}
    A very fancy sidebar
\end{RpgSidebar}
```

A basic filigree-wrapped tcbox.

Fancy Sidebar

A very fancy sidebar

RpgSecrets

Game Masters often need notes which their players should not see. The `RpgSecret` environment allows the same document to serve both purposes, by marking text as secret and choosing whether to reveal it.

`\begin{ RpgSecret }  [<options>]{<public-text>}`

Marks the body of the environment as secret. Its appearance depends on the `ShowSecrets` switch (page 17), which is off by default:

Switch	Display
Off	Only the (optional) `public-text` is shown, as ordinary text, with nothing to suggest that anything has been withheld.
On	The secret text is shown in a highlighted box, labelled as hidden. If `public-text` was given, it is shown in the lower part of the box, labelled as the publicly available version.

Secrets are revealed for the remainder of the document with `\RpgSwitch{ ShowSecrets }{ on }`. The following options are available:

Option	Effect	Default
`color`	The colour of the box's frame.	`SecretColor`
`label-public`	The label above the public text.	'Normal text'
`label-secret`	The label above the secret text.	'Hidden text'
`switch`	The switch which controls this secret. Using a different switch allows separate groups of secrets to be revealed independently.	`ShowSecrets`

The labels are typeset in the `\RpgFontSecretTitle` (page 68) font element.

RpgSecret: Hidden

```
\RpgSwitch{ShowSecrets}{false}

\lorumIpsumOne%Snippets of standard lorum ipsum
\begin{RpgSecret}{}
    The players don't know it yet, but all this
      pseudo-latin text is a hint that devils
      are involved!
\end{RpgSecret}
\lorumIpsumTwo{}
\begin{RpgSecret}{
    Logos kykloforia, thelxis kai ethos,
      tyrannis kai epilysis. Aei thera plaka,
      kyrios meion egestas, anagkas proin eira
      kai thera.
    }

    This bit of text is noteworthy - it's been
      added in after the fact
\end{RpgSecret}
\lorumIpsumThree
```

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Logos kykloforia, thelxis kai ethos, tyrannis kai epilysis. Aei thera plaka, kyrios meion egestas, anagkas proin eira kai thera. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

RpgSecret: Revealed

```
%%exactly the same as above, but reveal the
  secrets!
\RpgSwitch{ShowSecrets}{true}

\lorumIpsumOne%Snippets of standard lorum ipsum
\begin{RpgSecret}{}
   The players don't know it yet, but all this
     pseudo-latin text is a hint that devils
     are involved!
\end{RpgSecret}
\lorumIpsumTwo{}
\begin{RpgSecret}{
   Logos kykloforia, thelxis kai ethos,
     tyrannis kai epilysis. Aei thera plaka,
     kyrios meion egestas, anagkas proin eira
     kai thera.
   }

   This bit of text is noteworthy - it's been
     added in after the fact
\end{RpgSecret}
\lorumIpsumThree
```

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

Hidden text
The players don't know it yet, but all this pseudo-latin text is a hint that devils are involved!

Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

Hidden text
This bit of text is noteworthy - it's been added in after the fact

Normal text
Logos kykloforia, thelxis kai ethos, tyrannis kai epilysis. Aei thera plaka, kyrios meion egestas, anagkas proin eira kai thera.

Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

RpgMaps

Adventures often describe a location as a numbered list of areas, which correspond to the labels on a map. The `RpgMap` environment produces such a list, numbering and nesting the areas automatically.

`\begin{ RpgMap }` `[<options>]`

Begins a list of map areas, each of which is created with `\area`. Each area is given a heading, whose size depends on how deeply the map is nested: `\section*` at the top level, then `\subsection*`, and so on, down to `\subparagraph*`. These headings do not appear in the table of contents.

Areas are numbered according to their depth, cycling through numbers, capital letters and lower-case Roman numerals, so that the second area within the first area is `1B`, and its third sub-area is `1B-iii`. The following options are available:

Option	Effect	Default
`empty-prefix`	Text placed before the number of an unnamed area.	'Area '
`header-offset`	The number of levels by which to shrink the headings.	0
`prefix`	Text placed before the number of a named area.	'Area '
`title`	A title for the map, typeset as a heading one level larger than its areas. It is only used by the outermost map.	(empty)

`\RpgMapRef { label }`

`\RpgMapRef*`

Typesets the name of the labelled area. The starred form typesets its number instead. This is a wrapper around `\ref` and `\nameref`.

```
\RpgMapRef{map:dais} is area
  \RpgMapRef*{map:dais}, on
  \RpgPage{map:dais}.
```

The Dais is area 1A, on page 24.

Inside a Map

Inside a map environment, the following commands and environments are valid.

`\area [<label>]{<name>}`

Adds an area to the current map, much as `\item` adds an item to a list. Its heading is '`<prefix><number>: <name>`', or '`<empty-prefix><number>`' if no `name` is given. Any text which follows forms the description of the area.

If a `label` is given, the area may be referred to with `\RpgMapRef` or `\RpgPage`.

`\begin{ RpgArea } [<label>]{<name>}`

Adds an area to the current map, and begins a nested map within it, so that the body of the environment may contain sub-areas. It is equivalent to an `\area` followed by an `RpgMap` environment, and raises an error if used outside of a map.

Nested Map Areas

```
\begin{RpgMap}[header-offset=2]
    \begin{RpgArea}[map:hall]{Great Hall}
        A long and draughty hall.

        \area[map:dais]{The Dais} A
          raised stone platform.

        \area An unremarkable alcove.
    \end{RpgArea}

    \area[map:cellar]{Cellar} Damp and
      dark.
\end{RpgMap}
```

Area 1: Great Hall

A long and draughty hall.
Area 1A: The Dais. A raised stone platform.

Area 1B. An unremarkable alcove.

Area 2: Cellar

Damp and dark.

RpgCards

Cards are small, playing-card sized boxes, useful for handouts such as items, spells or abilities. If the content of a card is too long to fit, it is automatically continued onto further cards.

Cards are rarely used directly: they are most often created by the rule environments (page 25), which can display the same content either as normal text or as a card.

RpgCard example

```
\begin{RpgCard}[vmargin=0.1cm, hmargin=0.1cm,
  height=5.2cm, width=3.6cm]
    \raggedright\footnotesize This is an
      example card with lots of
      text\footnote{And maybe a footnote too?}.
    \lipsum[2]
\end{RpgCard}
```

This is an example card with lots of text*. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis.

Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

---

*And maybe a footnote too?

`\cardbreak {}`

Within an `RpgCard`, ends the current card and continues the text on the next, in the same way as `\pagebreak`. Outside of a card it has no effect, so it may safely be used in text which is only sometimes displayed as a card.

`\footnote { text }`

Within an `RpgCard`, `\footnote` is redefined so that its text appears at the end of the card's contents, below a horizontal rule. **Only one footnote may appear in each card**: if more than one is used, only the last is shown. Card footnotes do not affect the document's footnote numbering.

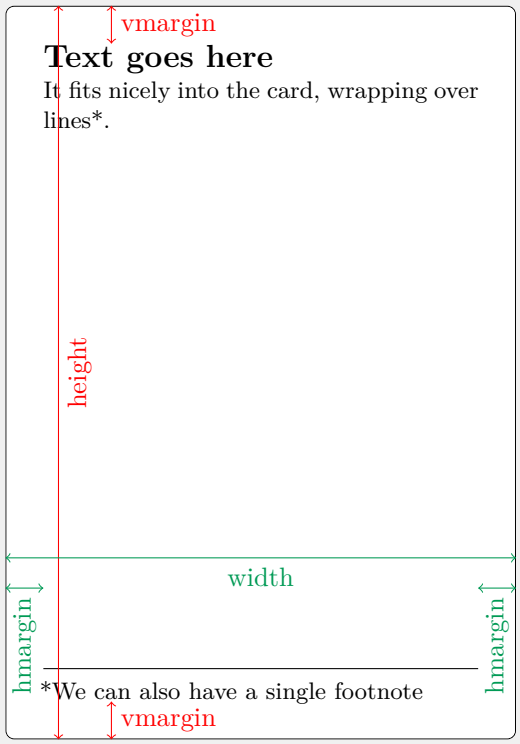
`\begin{ RpgCard } [<options>]`

Typesets its contents in one or more cards, placed side by side. The text is typeset in the `\RpgFontCardBody` (page 68) font element. The following options are available:

Option	Effect	Default
`cardsep`	The horizontal space between consecutive cards.	`5pt`
`color`	The colour of the card's background.	`white`
`footnote`	The symbol used to mark the card's footnote.	`*`
`height`	The outer height of each card.	`9.7cm`
`hmargin`	The horizontal space between the edge of the card and its text. It must be less than half of the `width`.	`0.2cm`
`opacity`	The opacity of the card's background.	`1`
`vcenter`	An alias of `vcentre`.	off
`vcentre`	If present, centres the text vertically within each card.	off
`vmargin`	The vertical space between the edge of the card and its text. It must be less than half of the `height`.	`0.2cm`
`width`	The outer width of each card.	`6.75cm`

Cards cannot be given a title, and cannot be nested within one another.

```
\begin{tikzpicture}%wrap in tikz so we
  can draw dimensions
\node[anchor=south west,inner
  sep=0pt,outer sep=0pt] at (0,0)
{
    \begin{RpgCard}[hmargin=0.5cm,
      vmargin=0.5cm, height=9.7cm,
      width=6.75cm]
        \subsubsection{Text goes here}
            It fits nicely into the card,
              wrapping over
              lines\footnote{We can also
              have a single footnote}.
    \end{RpgCard}\ignorespaces
};
%% Draw the dimenions on top; use macros
  to save space!
\heightLines{red}
\widthLines{ForestGreen}

\end{tikzpicture}
```



`\RpgCardReset {}`

Restores the default options of all subsequent cards to their original values.

`\RpgCardSet { options }`

Sets the default options of all subsequent cards. Options given to an individual `RpgCard` take priority over these defaults.

Rule Environments

The environments described so far are independent of any particular game system. Some elements, however, depend on the rules of the game being written for: a monster's statistics in one system look nothing like those in another. `RPGTeX` therefore provides four ***rule environments***, whose properties and appearance are defined by the theme:

Environment	Purpose
`RpgFeat`	Feats, talents and other character features.
`RpgItem`	Items and equipment.
`RpgSpell`	Spells and similar abilities.
`RpgStat`	Statistics blocks for creatures and characters.

Each may be displayed either as normal text or as a card, controlled by a switch. Since the available properties depend on the theme, consult the documentation of the active theme for the details of each.

\RpgGetKey *(Variable)*

The value given to a property of the current rule environment, such as \RpgGetKey{ rarity }. It may only be used within a rule environment.

```
\begin{RpgItem}{<name>}[<key> = <value>]
   The value of the key is \RpgGetKey{<key>}.
\end{RpgItem}
```

\begin{ RpgIfEnv } { switch }[<options-on>]{ env-on }[<options-off>]{ env-off }

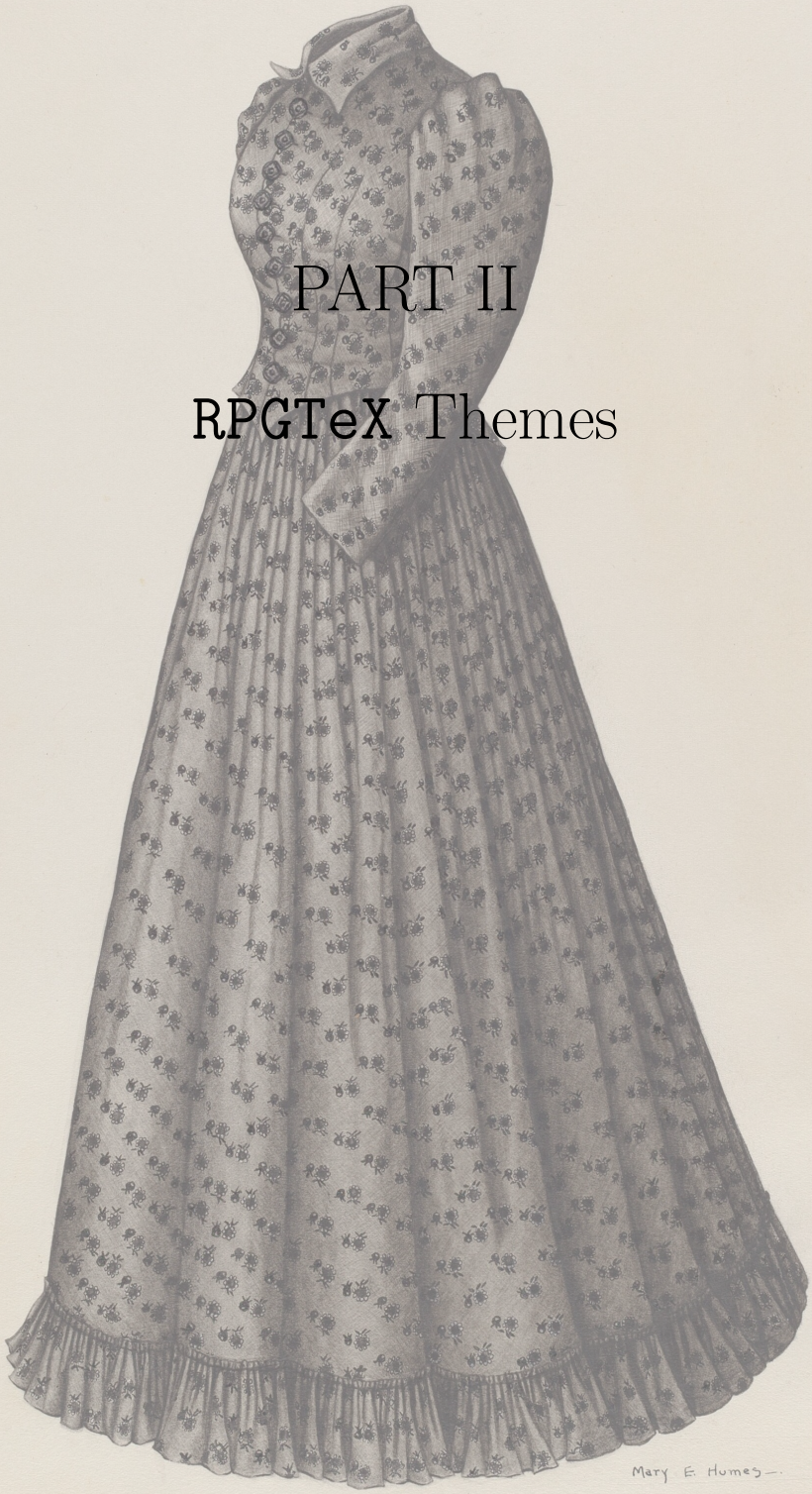
Begins the environment `env-on` (with `options-on`) if `switch` is on, and `env-off` (with `options-off`) otherwise. Any arguments which follow are passed to the chosen environment, so both environments must take the same arguments. This is the mechanism by which the rule environments switch between text and card modes, and it may be used to create similar switchable environments.

\begin{ Rpg[X] } [<card-options>]{ name }[<properties>]

Typesets the rule element 'X' called `name`, with the given `properties` (a list of key-value pairs defined by the theme). Its display depends on the corresponding switch (page 17):

Environment (X)	Switch
RpgFeat	UseFeatCard
RpgItem	UseItemCard
RpgSpell	UseSpellCard
RpgStat	UseStatCard

If the switch is on, the element is displayed as an `RpgCard` (page 24), to which the `card-options` are passed. Otherwise (the default), it is displayed as normal text, and the `card-options` are ignored.



PART II

RPGTeX Themes

Chapter 6: `default` Theme

As the name suggests, the `default` theme is the theme which is loaded if no arguments are passed to `theme` when loading the `RPGTeX` package. As such, it is largely a re-statement of the default LaTeX values. This is important, as most other themes begin by first re-inserting the default theme; thereby resetting everything into a 'known state'. This means that they only need to define formatting which differs from the default.

The majority of this document has been typeset in the rpgbook class (page 6) using the default theme.

Appearance

Fonts

If the `font` option is active, the default theme defines a single font family, `\lrfont`: Latin Modern Roman, the standard LaTeX font. For consistency with the rest of the library (which uses fontspec-defined fonts), we have used an OTF version of Latin Modern Roman, which is available on all systems with a TeX Live installation. It is used for the body text and the headings. The titles and headers of tables, the titles of tips and sidebars, and the section headings of statblocks instead use the standard sans-serif family, Latin Modern Sans.

`\lrfont {}`

Applies the Latin Modern Roman font-family (extended with a small-caps font; Latin Modern Roman Caps) defined by the default theme

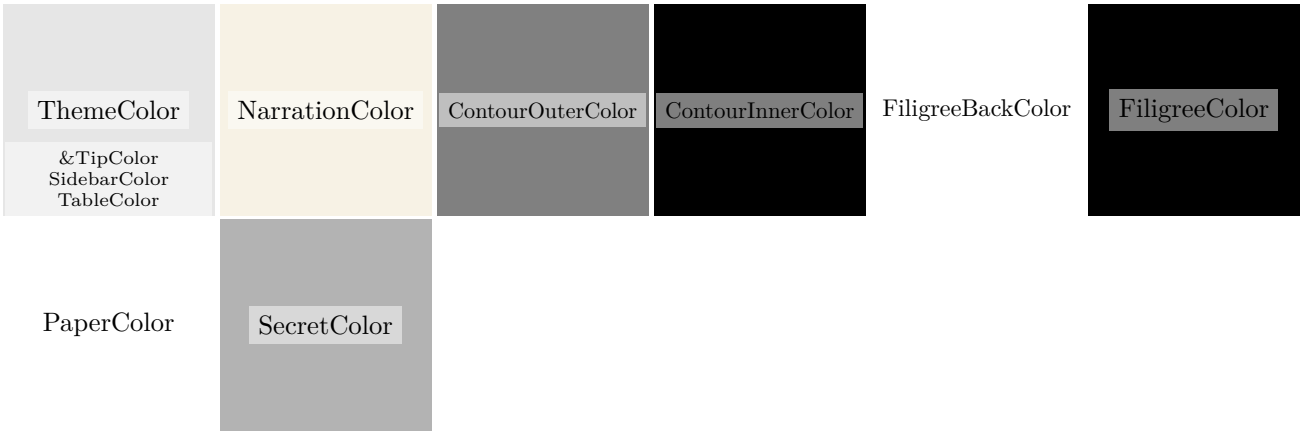
The values assigned to the font elements are as follows (note that `\normalfont` is aliased to the value of `\RpgFontBody` (page 67)).

Element	FontFamily	FontStyle	Example
`\RpgFontBody`	`\lrfont`		Lorum ipsum
`\RpgFontEmphasis`	`\normalfont`	`\bfseries \slshape`	***Lorum ipsum***
`\RpgFontTitle`	`\normalfont`	`\Huge`	Lorum ipsum
`\RpgFontSubTitle`	`\normalfont`	`\Large`	Lorum ipsum
`\RpgFontPart`	`\normalfont`	`\Huge`	Lorum ipsum
`\RpgFontChapter`	`\normalfont`	`\Huge \bfseries`	**Lorum ipsum**
`\RpgFontSection`	`\normalfont`	`\huge \bfseries`	**Lorum ipsum**
`\RpgFontSubsection`	`\normalfont`	`\Large \bfseries`	**Lorum ipsum**
`\RpgFontSubsubsection`	`\normalfont`	`\large \bfseries`	**Lorum ipsum**
`\RpgFontParagraph`	`\normalfont`	`\bfseries \slshape`	***Lorum ipsum***
`\RpgFontSubparagraph`	`\normalfont`	`\slshape`	*Lorum ipsum*
`\RpgFontTableTitle`	`\sffamily`	`\bfseries \large`	**Lorum ipsum**
`\RpgFontTableHeader`	`\sffamily`	`\bfseries`	**Lorum ipsum**
`\RpgFontTableBody`	`\normalfont`	`\small`	Lorum ipsum
`\RpgFontTipTitle`	`\sffamily`	`\bfseries`	**Lorum ipsum**
`\RpgFontTipBody`	`\normalfont`	`\small`	Lorum ipsum
`\RpgFontSidebarTitle`	`\sffamily`	`\bfseries \normalsize`	**Lorum ipsum**
`\RpgFontSidebarBody`	`\normalfont`	`\small`	Lorum ipsum
`\RpgFontNarration`	`\normalfont`	`\small`	Lorum ipsum
`\RpgFontAbstractTitle`	`\Large`	`\scshape \bfseries`	LORUM IPSUM
`\RpgFontAbstractBody`	`\normalfont`	`\small \slshape`	*Lorum ipsum*
`\RpgFontSecretTitle`	`\normalfont`	`\bfseries`	**Lorum ipsum**
`\RpgFontCardTitle`	`\normalfont`	`\bfseries \Large`	**Lorum ipsum**
`\RpgFontCardHeader`	`\normalfont`	`\it`	*Lorum ipsum*
`\RpgFontCardBody`	`\normalfont`	`\small`	Lorum ipsum
`\RpgFontTocPart`	`\normalfont`	`\Large \bfseries`	**Lorum ipsum**
`\RpgFontTocChapter`	`\normalfont`	`\large`	Lorum ipsum

\RpgFontTocSection	\normalfont	\normalsize	Lorum ipsum
\RpgFontStatBlockTitle	\normalfont	\bfseries \LARGE	**Lorum ipsum**
\RpgFontStatBlockBody	\normalfont	\small	Lorum ipsum
\RpgFontStatBlockSection	\sffamily	\large	Lorum ipsum
\RpgFontFooter	\normalfont	\scriptsize	Lorum ipsum
\RpgFontPageNumber	\normalfont	\scriptsize	Lorum ipsum
\RpgFontDropCap	\normalfont		Lorum ipsum
\RpgFontDropCapAfter	\normalfont	\slshape	*Lorum ipsum*

Colors

The default theme colors are largely monochromatic:



Backgrounds & Footers

The default theme defines no background or headers (and clears any existing ones). The footer consists of the chapter name and a page number, using the appropriate fonts (\RpgFontFooter (page 68) and \RpgFontPageNumber (page 68)).

Text Boxes

The default text boxes are very simple in design:

The RpgSidebar

Has minor decorations at the top and bottom to distinguish it from an RpgTip

RpgNarration is also simple; but uses the narration color

The RpgTip

Almost entirely undecorated

The filigree decorates each corner with curved strokes.

The filigree frame (page 21) draws four nested, curved strokes across each corner of the box, in FiligreeColor. The strokes reach from 0.2 to 1.4 times \RpgFiligreeSize (page 11) along each edge, so the design grows and shrinks with that size.

Title & Part Pages

The title of a document is created by \maketitle (page 12), in one of two styles: a compact title or a cover page. The class chooses which is used (page 5), and a designer may change it with \RpgThemeTitleMode (page 70). The theme decides how each style looks.

Compact Title

The compact title is used by the rpghandout and rpgdeck classes. The default theme centres the title (in \RpgFontTitle (page 67)) at the top of the page, followed by the subtitle and the author (both in

\RpgFontSubTitle (page 67)). In a two-column document, the title spans both columns. Since the compact title is part of the text, it can be shown here:

A Compact Title

```
\RpgThemeTitleMode{header}
\title{The Sunken Temple}
\subtitle{An adventure for four players}
\author{A. Writer}
\RpgMakeTitle
```

The Sunken Temple
An adventure for four players
A. Writer

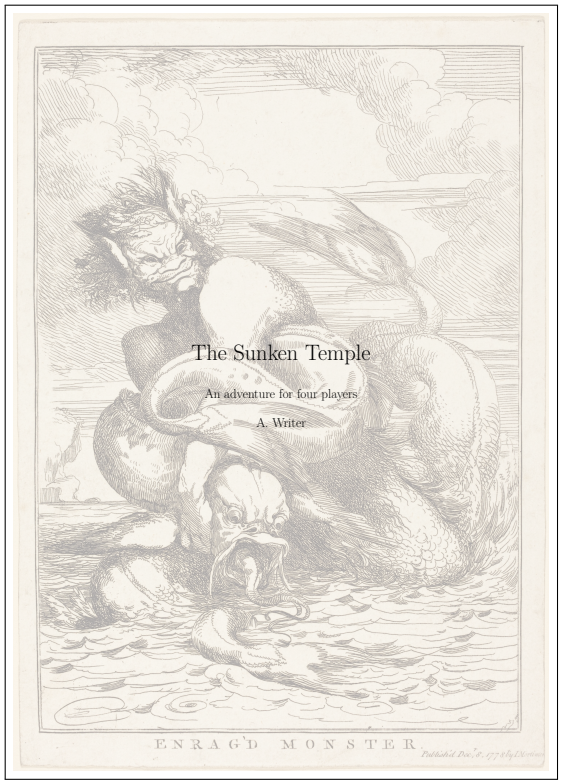
Cover Page

The cover page is used by the rpgbook class. The default theme places the title, subtitle, author and date in a column at the centre of a page of their own, in the same fonts as the compact title. If a cover image has been set with \cover (page 12), it fills the page behind the text, faded according to \RpgCoverTransparency (page 13). The first page of this manual is a default-theme cover page, with a typewriter as its cover image.

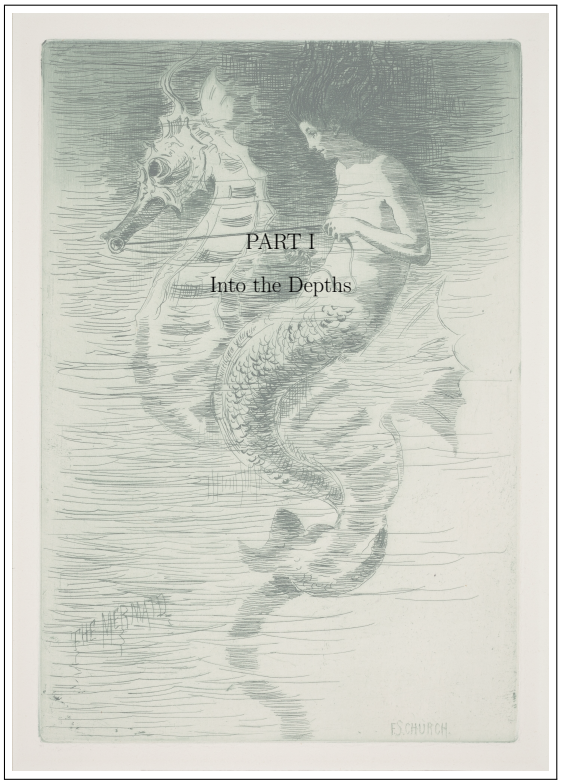
Part Pages

In classes which have parts, such as rpgbook, \part (page 13) produces a page of its own. The default theme places the word 'Part' and the part's number in capitals, with the name of the part below, centred a little above the middle of the page, in \RpgFontPart (page 67). An image may be placed behind the text, either for a single part (with the optional argument of \part) or for every part (with \RpgDefaultPartImage (page 13)), faded according to \RpgPartTransparency (page 13). The pages which begin each part of this manual are default-theme part pages.

The cover and part pages of a short example document, typeset with the default theme, are:



Cover page



Part page

Section Headers

Only parts and chapters[1] are numbered; all other section/subsections (etc.) are unnumbered. Each level uses its own font element, with some changes to the spacing, and two changes to the standard layout:

- A chapter heading is a single line, 'Chapter 3: Title', rather than the number and title on separate lines. In the appendices, 'Chapter' is replaced by 'Appendix'.
- Paragraph and subparagraph headings are run into the start of their text, and followed by a full stop. Subparagraph headings are also indented.

[1] In rpgbook, or another class which supports chapters and parts.

Section Headers

```
\section*{Example} %starred so not added to
  toc; otherwise the same
   \subsection{Another Example}
       \subsubsection{More Examples!}
           \paragraph{A Paragraph} Its text
             runs on from the heading.

           \subparagraph{A Subparagraph} As
             does this one.
```

Example

Another Example

More Examples!

A Paragraph. Its text runs on from the heading.
A Subparagraph. As does this one.

Table of Contents

The table of contents lists only parts, chapters and sections. Each part is listed as 'Part I: Title', with a rule beneath it, and each chapter as '3: Title'. Sections are indented beneath their chapter, with a dotted line leading to their page number. Parts and chapters are not given page numbers, except in the appendices, where each chapter is listed as 'Appendix A: Title', with its page number. The contents of this manual are typeset in the default theme.

RpgDice

The default RpgDice format is $ndx + m$, and if the modifier is zero, it is omitted:

Dice

```
\RpgDice{3d8 -5 +2}

\RpgDice{d8 + 0}
```

3d8-3
d8

The usual computing and collating of modifiers into a single value apply.

RpgItems

The default theme gives `RpgItem` (page 25) two properties:

Property	Effect	Default
`description`	A short summary of the item, shown in italics beneath its name.	*(none)*
`image`	The path to an image of the item, shown beneath its name and description, at most 3cm tall.	*(none)*

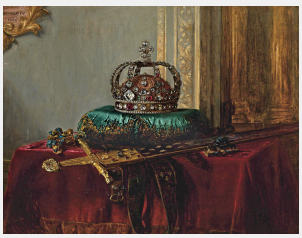
In text mode, the name of the item is typeset as a `\subsection`, followed by the description, the image and the body:

RpgItem: Text Mode

```
\begin{RpgItem}{Joyeuse}[
   description={The Sword Jewellous},
   image={img/joyeuse},
]
   The sword of Charlemagne; this jewelled
     sword gives you +3 to heroism checks.
\end{RpgItem}
```

Joyeuse

The Sword Jewellous



The sword of Charlemagne; this jewelled sword gives you +3 to heroism checks.

In card mode, which is turned on with `\RpgSwitch{ UseItemCard }{ on }` (page 18), the name, description and image are centred at the top of the card, in `\RpgFontCardTitle` (page 68) and `\RpgFontCardHeader` (page 68), and the body follows in `\RpgFontCardBody` (page 68):

RpgItem: Card Mode

```
\RpgSwitch{UseItemCard}{on}
\begin{RpgItem}[width=5cm, height=7cm]{Joyeuse}[
    description={The Sword Jewellous},
    image={img/joyeuse},
]
    The sword of Charlemagne; this jewelled
      sword gives you +3 to heroism checks.
\end{RpgItem}
\RpgSwitch{UseItemCard}{off}
```

Joyeuse

The Sword Jewellous

The sword of Charlemagne; this jewelled sword gives you +3 to heroism checks.

RpgFeats

The default theme gives `RpgFeat` (page 25) a single property:

Property	Effect	Default
`requires`	The prerequisites for gaining the ability, shown as 'Requires: …' in `\RpgFontCardHeader` (page 68).	*(none)*

In text mode, the name of the feat is typeset as a `\subsection`, followed by its prerequisites (if any) and the body:

RpgFeat: Text Mode

```
\begin{RpgFeat}{Hyperattack}[requires={Mega-attack}]
    You can unleash superhuman speed against
      your enemies. Once per day, make
      \RpgDice{3d10} additional attacks.
\end{RpgFeat}
```

Hyperattack

Requires: Mega-attack

You can unleash superhuman speed against your enemies. Once per day, make 3d10 additional attacks.

In card mode, which is turned on with `\RpgSwitch{ UseFeatCard }{ on }` (page 18), the prerequisites are placed at the bottom of the card instead:

RpgFeat: Card Mode

```
\RpgSwitch{UseFeatCard}{on}
\begin{RpgFeat}[width=5cm,
  height=6cm]{Hyperattack}[requires={Mega-attack}]
    You can unleash superhuman speed against
      your enemies. Once per day, make
      \RpgDice{3d10} additional attacks.
\end{RpgFeat}
\RpgSwitch{UseFeatCard}{off}
```

Hyperattack

You can unleash superhuman speed against your enemies. Once per day, make 3d10 additional attacks.

Requires: Mega-attack

RpgSpells & RpgStats

The default theme does not configure `RpgSpell` or `RpgStat` (page 25), since spells and statblocks depend so heavily on the rules of a particular game. They have no properties, and use the formats built into `RPGTeX`: the name is typeset as a `\subsection`, followed by the body, both in text mode and on a card. Themes for a particular game, such as `dnd`, define their own.

Chapter 7: dnd Theme

The dnd theme is designed to mimic the appearance of the *Dungeons & Dragons* sourcebooks. Most of its design is derived from the original DnD-5e-LaTeX-Template package, with some additional updates in line with the 2024 rules. It begins by loading the default theme (page 28), and so only changes the elements described below.

Appearance

Fonts

If the font option is active, the dnd theme defines four font families, all of which are part of TeX Live:

`\gillius {}`

Gillius ADF No2, a light sans-serif font, used for subtitles and for the body text of tables, tips, sidebars and narration.

`\gyreBonum {}`

TeX Gyre Bonum, a Bookman-style serif font, used for the main body text and statblocks.

`\kepler {}`

Kp Sans, used for the titles of tables, tips and sidebars, statblock sections and part subtitles.

`\keplerserif {}`

Kp Roman, used in small capitals for headings, emphasis, card titles and the table of contents.

Drop capitals use the decorative ROYAL initials, from the cfr-initials package. The theme also creates two font elements of its own: part-subtitle (\RpgFontPartSubtitle), for the name of a part on its part page, and stat-block-legendary (\RpgFontStatBlockLegendary), for the explanatory text of a statblock's legendary actions.

The values assigned to the font elements are as follows (note that \normalfont is aliased to the value of \RpgFontBody (page 67)):

Element	FontFamily	FontStyle	Example
\RpgFontBody	\gyreBonum		Lorum ipsum
\RpgFontEmphasis	\keplerserif	\bfseries \scshape \larger [1]	LORUM IPSUM
\RpgFontTitle	\normalfont	\bfseries \fontsize {30}{50}\selectfont \color {TitleRed}	Lorum ipsum
\RpgFontSubTitle	\gillius	\LARGE \color {TitleRed}	Lorum ipsum

\RpgFontPart	\normalfont	\color {TitleRed}\bfseries \fontsize {50}{30}\selectfont	Lorum ipsum
\RpgFontChapter	\normalfont	\titlemode \fontsize {30}{50}\selectfont	Lorum ipsum
\RpgFontSection	\normalfont	\titlemode \Huge	Lorum ipsum
\RpgFontSubsection	\normalfont	\titlemode \huge	Lorum ipsum
\RpgFontSubsubsection	\normalfont	\titlemode \Large	Lorum ipsum
\RpgFontParagraph	\normalfont	\bfseries \slshape	***Lorum ipsum***
\RpgFontSubparagraph	\normalfont	\bfseries \slshape	***Lorum ipsum***
\RpgFontTableTitle	\kepler	\bfseries \large \scshape	LORUM IPSUM
\RpgFontTableHeader	\kepler	\bfseries	Lorum ipsum
\RpgFontTableBody	\gillius	\small	Lorum ipsum
\RpgFontTipTitle	\kepler	\bfseries \scshape	LORUM IPSUM
\RpgFontTipBody	\gillius	\small	Lorum ipsum
\RpgFontSidebarTitle	\kepler	\bfseries \scshape	LORUM IPSUM
\RpgFontSidebarBody	\gillius	\small	Lorum ipsum
\RpgFontNarration	\gillius	\small	Lorum ipsum
\RpgFontAbstractTitle	\Large	\scshape \bfseries	LORUM IPSUM
\RpgFontAbstractBody	\normalfont	\small \slshape	*Lorum ipsum*
\RpgFontSecretTitle	\normalfont	\bfseries	**Lorum ipsum**
\RpgFontCardTitle	\keplerserif	\color {TitleRed}\large \bfseries	**Lorum ipsum**
\RpgFontCardHeader	\normalfont	\normalfont \scshape \footnotesize	LORUM IPSUM
\RpgFontCardBody	\normalfont	\small	Lorum ipsum
\RpgFontTocPart	\normalfont	\LARGE \keplerserif \scshape \color {TitleRed}	Lorum ipsum
\RpgFontTocChapter	\normalfont	\Large \keplerserif \scshape \color {TitleRed}	Lorum ipsum
\RpgFontTocSection	\normalfont	\normalsize	Lorum ipsum
\RpgFontStatBlockTitle	\scshape	\bfseries \LARGE \color {StatblockHeading}	Lorum ipsum
\RpgFontStatBlockBody	\gyreBonum	\small \color {black}	Lorum ipsum
\RpgFontStatBlockSection	\kepler	\color {StatblockHeading}\s \large	LORUM IPSUM
\RpgFontFooter	\normalfont	\color {FooterGold}\scriptsize	Lorum ipsum
\RpgFontPageNumber	\normalfont	\color {FooterGold}\scripts	Lorum ipsum
\RpgFontDropCap	\Royal		L
\RpgFontDropCapAfter	\normalfont	\slshape	*Lorum ipsum*
\RpgFontPartSubtitle	\kepler	\color {TitleRed}\Large \bfseries	Lorum ipsum
\RpgFontStatBlockLegendary	\normalfont	\it \scriptsize \color {gray!90}	*Lorum ipsum*

Colors

The dnd theme assigns the following values to the standard colors:

It also defines a number of colors of its own:

The colors from `BrGreen` to `DmgPatina` match the tables and sidebars of the corresponding parts of the Basic Rules, the *Player's Handbook* and the *Dungeon Master's Guide* when set as the `ThemeColor` (page 12). Rather than setting them by hand, the theme steps through a cycle of these colors, moving to the next one at the start of each part, so that each part of a document has its own color. The following commands control the cycle:

`\RpgAdvanceColors {}`

Moves to the next color in the cycle immediately, and sets it as the `ThemeColor`. This is called automatically at the start of each part.

`\RpgSetDMGColors {}`

Sets the cycle to the colors of the *Dungeon Master's Guide*, followed by those of the *Player's Handbook*, and moves to the first of them.

`\RpgSetPHBColors {}`

Sets the cycle to the colors of the *Player's Handbook*, followed by those of the *Dungeon Master's Guide*, and moves to the first of them. This is the cycle used by default.

`\RpgThemeSetColorCycle { colors }`

Sets the cycle to the comma-separated list of `colors`, and moves to the first of them.

Backgrounds & Footers

Each page is given a plain background in `Papyrus`, the `PaperColor` of the theme. A paper texture may be used instead:

`\RpgSetDndPaper {}`

Sets the background of each subsequent page to the theme's paper texture image.

This effect is demonstrated on this page (and the subsequent pages of the dnd theme).

The footer is decorated with a scroll across the bottom of the page, which is mirrored on alternate pages in two-sided documents[1]. The name of the chapter is placed above the scroll, in `\RpgFontFooter` (page 68), and the page number sits in a notch in the scroll, in `\RpgFontPageNumber` (page 68).

Text Boxes

The `dnd` theme gives the text boxes the following appearance:

The RpgSidebar
Horizontal 'ribbons' and sharp corners differentiate it from RpgTip.

The RpgTip
Almost entirely undecorated, with rounded corners.

RpgNarration has red 'bars' on either side, ending in dots.

Filigree

The `dnd` filigree frame (page 21) mimics the decoration around the class tables of the *Player's Handbook*: a double border, with small outcrops at each corner. It may be applied to any `tcolorbox`, including tables:

Filigree

```
\begin{RpgFiligree}
   Some decorative text
\end{RpgFiligree}

\vspace{1cm}

\begin{RpgTable}[filigree]{lX}
   Header 1 & Header 2
   \\
   Body 1 & Body 2
   \\
   Body 3 & Body 4
\end{RpgTable}
```

Some decorative text

Header 1	Header 2
Body 1	Body 2
Body 3	Body 4

[1] This document is set to use the `oneside` option, so the footnotes do not alternate

Title & Part Pages

The compact title is the same as in the default theme (page 29), but uses the fonts of the dnd theme:

A Compact Title

```
\RpgThemeTitleMode{header}
\title{The Sunken Temple}
\subtitle{An adventure for four players}
\author{A. Writer}
\RpgMakeTitle
```

The Sunken Temple

An adventure for four players

A. Writer

On the cover page, the following formatting is applied:

- The title is styled using \RpgContour (page 15) with the colours set by TitleInner and TitleOuter (default: white and black)
- A red diamond is displayed beneath the title
- The subtitle and author are styled using \RpgContour with the colours set by SubtitleInner and SubtitleOuter (default: black and white).

On a part page, the word 'Part' and the part's number are placed in a badge at the top of the page, with the name of the part beneath it in \RpgFontPartSubtitle. The badge has two styles:

`\RpgThemeSetDMGParts {}`

Draws the badge on subsequent part pages as a shield, in the style of the *Dungeon Master's Guide*.

`\RpgThemeSetPHBParts {}`

Draws the badge on subsequent part pages as a semicircle, in the style of the *Player's Handbook*. This is the style used by default.

The cover and part pages of a short example document, typeset with the dnd theme, are:

Cover page

Part page

Section Headers

As in the default theme, only parts and chapters are numbered. Headings are set in small capitals, in `TitleRed`. Chapter headings are outlined with `\RpgContour` (page 15), and subsection headings are underlined with a rule in `TitleGold`:

Section Headers

```
\section*{Example} %starred so not added to
  toc; otherwise the same
   \subsection{Another Example}
      \subsubsection{More Examples!}
         \paragraph{A Paragraph} Its text
           runs on from the heading.

         \subparagraph{A Subparagraph} As
           does this one.
```

Example

Another Example

More Examples!

A Paragraph. Its text runs on from the heading.
A Subparagraph. As does this one.

Table of Contents

The table of contents is laid out as in the default theme (page 28), but the entries for parts and chapters are set in small capitals, in `TitleRed`, and the rule beneath each part is in `TitleGold`.

RpgDice

Following the style of D&D statblocks, `\RpgDice` (page 15) shows the average of the roll, followed by the roll itself in brackets. A roll without a number of dice, such as `d8`, is shown as `1d8`:

Dice

```
\RpgDice{3d8 -5 +2}

\RpgDice{d8 + 0}
```

10 (3d8-3)
4 (1d8)

RpgItems

The `dnd` theme gives `RpgItem` (page 25) the following properties:

Property	Effect	Default
`description`	A short summary of the item, shown in small capitals beneath its name in card mode, in place of its rarity and type.	*(none)*
`image`	The path to an image of the item, shown beneath its name in card mode, at most 3cm tall.	*(none)*
`rarity`	The rarity of the item, such as `Rare`.	*(none)*
`requires-attunement`	Whether the item requires attunement. Given as a flag, it adds 'Requires attunement'; given a value, such as by `a paladin`, the value is added after it.	`none`
`type`	The type of the item, such as `weapon (sword)` or `wondrous item`.	*(none)*

In text mode, the name of the item is typeset as a `\subsubsection`, followed by a line in italics giving its rarity, type and attunement, and then the body:

RpgItem: Text Mode

```
\begin{RpgItem}{Joyeuse}[
    rarity = Rare,
    type = weapon (sword),
    requires-attunement, %% a flag, with no
      value
]
    The sword of Charlemagne; this jewelled
      sword gives you +3 to heroism checks.
\end{RpgItem}
```

Joyeuse

Rare weapon (sword). Requires attunement
The sword of Charlemagne; this jewelled sword gives you +3 to heroism checks.

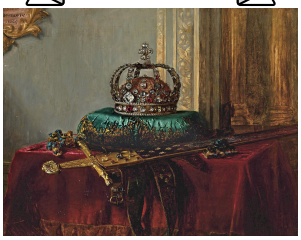
In card mode, which is turned on with `\RpgSwitch{ UseItemCard }{ on }` (page 18), the name is placed on a ribbon, which shrinks the text if it would not otherwise fit. The description (or the rarity and type) is placed above it, and the image, attunement and body below:

RpgItem: Card Mode

```
\RpgSwitch{UseItemCard}{on}
\begin{RpgItem}[width=5.5cm,
  height=8cm]{Joyeuse}[
    rarity = Rare,
    type = weapon (sword),
    requires-attunement = {by a paladin},
    image = {img/joyeuse},
]
    The sword of Charlemagne; this jewelled
      sword gives you +3 to heroism checks.
\end{RpgItem}
\RpgSwitch{UseItemCard}{off}
```

Rare weapon (sword)

Joyeuse



Requires attunement by a paladin

The sword of Charlemagne; this jewelled sword gives you +3 to heroism checks.

RpgFeats

The dnd theme gives RpgFeat (page 25) the following properties:

Property	Effect	Default
prerequisite	The prerequisites for gaining the feat, shown as 'Prerequisite: ...'.	*(none)*
requires	The same as prerequisite, for compatibility with the default theme (page 32).	*(none)*

In text mode, the name of the feat is typeset as a \subsection, followed by its prerequisites in italics, and the body:

RpgFeat: Text Mode

```
\begin{RpgFeat}{Grappler}[requires={Strength 13
  or higher}]
    You've developed the skills necessary to
      hold your own in close-quarters
      grappling. You gain the following
      benefits:
    \begin{itemize}
        \item You have advantage on attack
          rolls against a creature you are
          grappling.
        \item You can use your action to try to
          pin a creature grappled by you.
    \end{itemize}
\end{RpgFeat}
```

Grappler

Prerequisite: Strength 13 or higher

You've developed the skills necessary to hold your own in close-quarters grappling. You gain the following benefits:

- You have advantage on attack rolls against a creature you are grappling.
- You can use your action to try to pin a creature grappled by you.

In card mode, which is turned on with \RpgSwitch{ UseFeatCard }{ on } (page 18), the prerequisites are centred at the bottom of the card instead:

RpgFeat: Card Mode

```
\RpgSwitch{UseFeatCard}{on}
\begin{RpgFeat}[width=5.5cm, height=7.5cm,
  color=DmgCoral]{Grappler}[prerequisite={Strength
  13 or higher}]
    You've developed the skills necessary to
      hold your own in close-quarters
      grappling. You have advantage on attack
      rolls against a creature you are
      grappling.
\end{RpgFeat}
\RpgSwitch{UseFeatCard}{off}
```

Grappler

You've developed the skills necessary to hold your own in close-quarters grappling. You have advantage on attack rolls against a creature you are grappling.

Prerequisite: Strength 13 or higher

RpgSpells

The dnd theme gives RpgSpell (page 25) the components of a D&D spell as properties:

Property	Effect	Default
area	The area of effect of the spell. If given, 'Area of Effect' is shown in place of 'Range', with the range in brackets after it.	*(none)*
casting-time	The time taken to cast the spell.	1 action
components	The verbal, somatic and material components of the spell.	VSM
duration	The duration of the spell.	Instantaneous
level	The level of the spell, as a whole number. Level 0 is shown as a cantrip.	0
range	The range of the spell.	self
school	The school of magic of the spell, such as Evocation.	*(none)*
source	The book in which the spell is found. If given, it is shown after the other components.	*(none)*

Any of these may contain a \footnote, although only a single footnote may be used on a card (page 24).

In text mode, the name of the spell is typeset as a \subsubsection, followed by its level and school in italics (such as '3rd level Evocation', or 'Evocation Cantrip'), a list of its components, and the body:

RpgSpell: Text Mode

```
\begin{RpgSpell}{Fire Bolt}[
    school = Evocation,
    level = 0,
    range = 120ft,
    components = VS,
    source = Free Basic Rules (2014),
]
    You hurl a mote of fire at a creature or
      object within range. On a hit, the target
      takes \RpgDice{1d10} fire damage.

    \RpgCantripScaling{1d10}{2d10}{3d10}{4d10}
\end{RpgSpell}
```

Fire Bolt

Evocation Cantrip

Casting time: 1 action
Range: 120ft
Components: VS
Duration: Instantaneous
Source: *Free Basic Rules (2014)*

You hurl a mote of fire at a creature or object within range. On a hit, the target takes 5 (1d10) fire damage.
This cantrip improves with your character level:

Level	1st	5th	11th	17th
Damage	1d10	2d10	3d10	4d10

In card mode, which is turned on with \RpgSwitch{ UseSpellCard }{ on } (page 18), the level, school and name are placed at the top of the card, above a grid of the casting time, range, components and duration. The source is placed at the bottom of the card:

RpgSpell: Card Mode

```
\RpgSwitch{UseSpellCard}{on}
\begin{RpgSpell}[width=6cm, height=9cm]{Cone of
  Cold}[
    school = Evocation,
    level = 5,
    range = self,
    area = 60ft cone,
    components = VSM\footnote{A small crystal
      or glass cone},
    source = Free Basic Rules (2014),
]
    A blast of cold air erupts from your hands.
      Each creature in the cone must make a
      Constitution saving throw, taking
      \RpgDice{8d8} cold damage on a failed
      save, or half as much on a successful one.

    \RpgSpellUpcast{1d8}
\end{RpgSpell}
\RpgSwitch{UseSpellCard}{off}
```

5th level Evocation

Cone of Cold

Casting time	Area of Effect
1 action	60ft cone (self)
Components	**Duration**
VSM*	Instantaneous

A blast of cold air erupts from your hands. Each creature in the cone must make a Constitution saving throw, taking 36 (8d8) cold damage on a failed save, or half as much on a successful one.

At higher levels. The damage increases by 1d8 for every spell level above 5th used to cast this spell.

Source: Free Basic Rules (2014)

*A small crystal or glass cone

The theme also provides two commands for the standard wording of D&D spells:

`\RpgCantripScaling [<name>]{ 1st }{ 5th }{ 11th }{ 17th }`

Typesets the sentence 'This cantrip improves with your character level', followed by a table of the values of the cantrip at levels 1, 5, 11 and 17. The `name` labels the row of the table, and is `Damage` by default.

`\RpgSpellUpcast [<text>]{ increase }[<levels>]`

Typesets the 'At higher levels' paragraph of a spell: the `text` ('The damage increases by', by default), followed by the `increase` for every `levels` spell levels (`1`, by default) above the level of the spell. Since it uses the level of the current spell, it should only be used within an `RpgSpell`:

Upcasting with Optional Arguments

```
\begin{RpgSpell}{Slow Time}[level = 3,
  school = Transmutation, duration = 1
  minute]
    Time slows around a creature of your
      choice.

    \RpgSpellUpcast[The duration
      increases by]{1 minute}[2]
\end{RpgSpell}
```

Slow Time

3rd level Transmutation

Casting time: 1 action
Range: self
Components: VSM
Duration: 1 minute

Time slows around a creature of your choice.

At higher levels. The duration increases by 1 minute for every 2 spell levels above 3rd used to cast this spell.

RpgStats

The dnd theme turns the RpgStat rules environment (page 25) into a D&D monster statblock matching those found in the Monster Manuals. A statblock has two halves, which are given in different ways:

- The *statistics* (hit points, armor class, ability scores, skills and so on) are highly structured, and are given as key-value arguments to the environment itself (page 42). The theme computes the modifiers, saves and skill bonuses from them.
- The *abilities* (traits, actions, reactions and so on) are freeform text, and are written in the body of the environment, with the help of the commands described on page 44.

A Complete Statblock

```
\begin{RpgStat}{Gorgenhar, the Devourer}[
    nickname = Gorgenhar,
    size = Large, type = Fiend, alignment =
      Chaotic Evil,
    ac = 17 (natural armor), hp =
      \RpgDice{10d10+30}, speed = 40ft,
    str = 20, dex = 14, con = 16, int = 11, wis
      = 13, cha = 18,
    str-save, con-save,
    skills = {perception, athletics},
      skills-expertise = {stealth},
    senses = darkvision 60ft, languages =
      Abyssal,
    damage-resistances = fire,
      condition-immunities = poisoned,
    cr = 7,
]
    \section{Traits}
        \trait{Fiery Aura} At the start of each
          of its turns, each creature within
          5ft of \name{} takes \RpgDice{1d6}
          fire damage.
    \section{Actions}
        \melee{Claw}[dmg=2d6+5,
          dmg-type=slashing, plus-dmg=1d6,
          plus-type=fire]
\end{RpgStat}
```

Gorgenhar, the Devourer

Large Fiend, Chaotic Evil

AC	17 (natural armor)	**Speed**	40ft
HP	85 (10d10+30)	**Initiative**	+2

	STR (20)	DEX (14)	CON (16)	INT (11)	WIS (13)	CHA (18)
MOD	+5	+2	+3	+0	+1	+4
SAVE	+8		+6			

Skills Stealth (+8), Perception (+4), Athletics (+8)
Damage Resistances fire
Condition Immunities poisoned
Senses darkvision 60ft, passive Perception 14
Languages Abyssal
Challenge 7 (3000xp, proficiency: +3)

Traits

Fiery Aura. At the start of each of its turns, each creature within 5ft of Gorgenhar takes 3 (1d6) fire damage.

Actions

Claw. *Melee (5ft):* +8 to hit. *On Hit:* 12 (2d6+5) slashing damage plus 3 (1d6) fire damage.

Design Philosophy

The design philosophy of the dnd theme RpgStat environment is that – wherever possible – the environment should do the computational work for the author. If they declare that the monster has a strength of 15, then this should be reflected in their save scores, associated skills and any strength-based attacks. This way, when they update the score to 18, everything else should follow suit.

This intrinsically links the dnd theme to the rules of D&D 5th edition.

Statistics

The following values may be provided to the RpgStat environment to define the key statistics. These are often not used as-is, and are instead used to derive and compute values which are typeset, see page 44.

Where a property has several names, they may be used interchangeably. If multiple values are given to the same key (or aliases thereof), the most-recently declared is used.

Description

Property	Effect	Default
alignment	The alignment of the creature, shown after its size and type.	Neutral

`armor-class` *also* `ac`, `AC`	The armor class, which may include an explanation, such as `15 (natural armor)`.	`10`
`hit-points` *also* `hp`, `HP`	The hit points, usually expressed as a dice expression. **Notes:** Valid dice expressions are resolved automatically with `\RpgDice`, otherwise printed as-is.	`1d8`
`initiative`	The initiative bonus, used as it is written. **Notes:** If it is not given, the Dexterity modifier is used.	*(computed)*
`nickname` *also* `name`	A shorter name for the creature, such as 'Gorgenhar' for 'Gorgenhar, the Devourer'. **Notes:** Use `\RpgStatName` (or `\name`) in the abilities to refer to the creature by this name.	*(the title)*
`size`	The size of the creature, shown beneath its name.	`Medium`
`speed`	The speed of the creature, used as it is written. **Notes:** Include the units and any other movement, such as `30ft, fly 60ft`.	`30ft`
`type`	The type of the creature, shown after its size.	`Creature`

Abilities & Saves

Property	Effect	Default
`str, dex, con, int, wis, cha` *also* `STR, DEX, CON, INT, WIS, CHA`	The ability scores, as whole numbers.	`10`
`<ability>-save`	Flags, such as `dex-save` (or `DEX-save`), which make the creature proficient in the corresponding saving throw. **Notes:** The proficient saves are shown in a row beneath the modifiers.	*(off)*
`proficiency-bonus` *also* `proficiency`, `pb`	The proficiency bonus, as a whole number. **Notes:** If it is not given, it is computed from the challenge rating (page 44).	*(computed)*

Skills & Details

Property	Effect	Default
`challenge` *also* `cr`, `CR`	The challenge rating, which may be a fraction, such as `1/4`. **Notes:** If it is not given, it is computed from the proficiency bonus, or is 0 if neither is given (page 44).	*(computed)*
`condition-immunities`	The conditions to which the creature is immune.	*(none)*
`damage-immunities`	The damage types to which the creature is immune.	*(none)*
`damage-resistances`	The damage types to which the creature is resistant.	*(none)*
`damage-vulnerabilities`	The damage types to which the creature is vulnerable.	*(none)*
`languages` *also* `language`	The languages the creature speaks or understands.	*(none)*
`passive-perception`	The passive Perception, used as it is written. **Notes:** If it is not given, it is 10 plus the Wisdom (Perception) bonus.	*(computed)*
`senses`	Any special senses, shown before the passive Perception.	*(none)*
`skills` *also* `skill`, `proficient`	A comma-separated list of the skills in which the creature is proficient. **Notes:** Each of the eighteen D&D skills is given its bonus automatically; anything else, such as `Acrobatics (+9)`, is shown as it is written.	*(none)*
`skills-expertise` *also* `expert`, `expertise`	A comma-separated list of the skills in which the creature has expertise. **Notes:** These skills are listed before the others, with twice the proficiency bonus.	*(none)*
`experience` *also* `xp`, `XP`	The experience points for defeating the creature. **Notes:** If it is not given, it is computed from the challenge rating.	*(computed)*

Appearance & Placement

Property	Effect	Default
`color`	The color of the statblock's rules. **Notes:** The headings are set in a darker shade of it, the frame in a lighter shade, and the tables in a pale shade. If it is not given, the theme's colors are used (page 52).	*(theme)*

`filigree`	A flag, which decorates the statblock with the filigree frame (page 36).	*(off)*
`filigree-match`	A flag, which decorates the statblock with the filigree frame, in the color of its frame.	*(off)*
`float`	If given, places the statblock in a float, with this placement (such as `t` or `h`).	*(none)*
`float-type`	The kind of float used, such as `figure*` for a statblock spanning both columns of a two-column document.	`figure`
`frame-color` *also* `outline-color`	The color of the frame, and of the filigree of `filigree-match`.	*(from `color`)*
`twocolumn`	A flag, which sets the contents of the statblock in two columns, for statblocks which span the width of the page.	*(off)*

The appearance and placement properties have no effect in card mode.

Derived Values

From these, several other values are automatically computed:

- The modifier of each ability score is computed in the usual way ($\lfloor \frac{\text{score}-10}{2} \rfloor$).
- The challenge rating (CR) and the proficiency bonus (PB) are coupled together:
  1. If neither is given, the CR is set to 0, and the PB is set to +2
  2. If only one is given, the other value is computed using $\text{PB} = 2 + \max(0, \lfloor(\text{CR} - 1)/4\rfloor)$
  3. If both are given, they are both displayed (even if they would contradict the established formula), and the provided PB is used in all calculations going forward.
- If the creature is declared proficient in a saving throw, a row of save bonuses (computed from the modifier + the proficiency bonus) is shown beneath the modifiers. Only the proficient scores are shown.
- The experience points are computed from the challenge rating, by an approximate relation which is accurate to the values in the Monster Manual to within 100xp up to challenge 16.
- If no value is provided to the 'initiative' field, the Dexterity modifier is used
- Skills (and expertises) can be declared as comma-separated lists:
  1. If the name matches (case-insensitive) one of the 18 D&D skills, the value is typeset with a bonus computed from the associated ability modifier plus the PB (or twice the PB for expertise). The associated ability is hardcoded (so Intimidation always uses the Charisma bonus).
  2. If the name does not match, it is typeset exactly. Custom bonuses can be given this way, as "Athletics (+18)" would not match "Athletics".
- The passive Perception is 10 plus the Wisdom (Perception) bonus, taking into account the assigned proficiency.

Names & Values

The following commands give the values of the current statblock, for use within its abilities:

`\RpgStatMod { ability }`

`\stat` The modifier of the given `ability` (`str`, `dex`, `con`, `int`, `wis` or `cha`), or the proficiency bonus (`proficiency`), with its sign, such as +3.

`\RpgStatName` *(Variable)*

`\name` The name of the creature, as used in its abilities: its `nickname`, if one was given, and otherwise its title.

`\RpgStatSetName { name }`

Changes the name given by `\RpgStatName`.

Abilities

The abilities of the creature are written in the body of the environment, using the commands below.

Short Names

Within an RpgStat, each of the following commands has a short name, given as its alias below: \action for \RpgStatAction, for example. The short names are only created if no command of that name already exists, so they never replace a command of the document's own. The one exception is \section, which is always replaced by \RpgStatSection within a statblock.

Sections & Actions

`\RpgStatAction [<cost>]{ name }`

`\action`

Begins a trait or action called `name`, followed by its description. The `cost` is the number of legendary actions it uses, and is shown as '(2 actions)'. It is also available as `\trait`.

`\RpgStatReaction { name }{ trigger }`

`\reaction`

Begins a reaction called `name`, followed by its `trigger` in italics, and its description. If the `trigger` is empty, it is left out.

`\RpgStatSection { title }`

`\section`

Begins a section of the statblock, such as 'Traits' or 'Actions', with its `title` in \RpgFontStatBlockSection (page 68) above a rule.

In addition to the standard sectioning command, we provide special commands which initiate a Legendary Actions or Mythic Monster section, including the introductory preamble text. These are found on page 49.

Sections, Traits & Reactions

```
\begin{RpgStat}{Guard}[ac=16 (chain shirt),
  hp=\RpgDice{3d8+3}, str=13, con=12]
    \section{Traits}
        \trait{Watchful} The guard can't be
          surprised while conscious.
    \section{Actions}
        \action{Halberd Sweep} The guard
          attacks each creature within 10ft.
    \section{Reactions}
        \reaction{Parry}{When hit by a melee
          attack} The guard adds 2 to its AC.
\end{RpgStat}
```

Guard

Medium Creature, Neutral

AC	16 (chain shirt)	**Speed**	30ft
HP	16 (3d8+3)	**Initiative**	+0

	STR (13)	DEX (10)	CON (12)	INT (10)	WIS (10)	CHA (10)
MOD	+1	+0	+1	+0	+0	+0

Senses passive Perception 10
Challenge 0 (0xp, proficiency: +2)

Traits

Watchful. The guard can't be surprised while conscious.

Actions

Halberd Sweep. The guard attacks each creature within 10ft.

Reactions

Parry. *When hit by a melee attack:* The guard adds 2 to its AC.

Attacks

`\RpgStatAttack { name }[<options>]`

`\attack`

Typesets an attack called `name` in the standard form, '*Melee (5ft):* +5 to hit. *On Hit:* ...'. The `options` are:

Option	Effect	Default
bonus	An extra bonus to hit.	0

distance	melee, ranged or both. **Notes:** \melee and \ranged set this for you.	melee
dmg	The damage of the attack, as dice, such as 1d6+3. **Notes:** Unlike to-hit, the modifier is not added here, but may be included in the calculation using \RpgStatMod, which allows numerical expressions.	1d4
dmg-type	The type of the damage.	bludgeoning
extra	Any further effect of a hit, added after the damage. **Notes:** Do not end it with a full stop, which is added for you.	*(none)*
modifier	The ability used by the attack: str, dex, con, int, wis or cha, or the same in capitals.	str
or-dmg	Alternative damage, such as for a weapon used with two hands.	*(none)*
or-type	The type of the alternative damage.	*(none)*
or-when	When the alternative damage applies, such as when used with two hands.	*(none)*
plus-dmg	Additional damage, added to the main damage.	*(none)*
plus-type	The type of the additional damage.	slashing
range	The range of a ranged attack, in feet. **Notes:** Used when distance is ranged or both. The units are added if missing.	20/60
reach	The reach of a melee attack, in feet. **Notes:** Used when distance is melee or both. The units are added if missing.	5
type	weapon or spell. **Notes:** A spell attack is shown as 'Melee Spell' or 'Ranged Spell'.	weapon

The displayed to-hit modifier is computed automatically from:

$$\text{to-hit} = \text{modifier} + \text{PB} + \text{bonus}$$

`\melee { name }[<options>]`

An alias for `\attack[distance=melee, <options>]{ name }`.

`\ranged { name }[<options>]`

An alias for `\attack[distance=ranged, <options>]{ name }`.

Attacks

```
\begin{RpgStat}{Bandit Captain}[str=15, DEX=16,
  cr=2,type={Humanoid (Elf)}]
    \section{Actions}
        \melee{Scimitar}[modifier=dex,
          dmg=1d6+\RpgStatMod{dex},
          dmg-type=slashing]
        \ranged{Dagger}[modifier=dex,
          range=20/60,
          dmg=1d4+2*\RpgStatMod{dex},
          dmg-type=piercing]
        \attack{Spear}[distance=both, reach=5,
          range=20/60,
          dmg=1d6+\RpgStatMod{str},
          dmg-type=piercing,
            or-dmg=1d6+2*\RpgStatMod{str},
              or-type=piercing, or-when={when
              used with two hands}]
        \melee{Flame Blade}[type=spell,
          modifier=cha, dmg=3d6, dmg-type=fire]
\end{RpgStat}
```

Bandit Captain

Medium Humanoid (Elf), Neutral

AC	10	**Speed**	30ft
HP	4 (1d8)	**Initiative**	+3

	STR (15)	DEX (16)	CON (10)	INT (10)	WIS (10)	CHA (10)
MOD	+2	+3	+0	+0	+0	+0

Senses passive Perception 10
Challenge 2 (400xp, proficiency: +2)

Actions

Scimitar. *Melee (5ft):* +5 to hit. *On Hit:* 6 (1d6+3) slashing damage.

Dagger. *Ranged (20/60ft):* +5 to hit. *On Hit:* 8 (1d4+6) piercing damage.

Spear. *Melee (5ft) or Ranged (20/60ft):* +4 to hit. *On Hit:* 5 (1d6+2) piercing damage, or 7 (1d6+4) piercing damage when used with two hands.

Flame Blade. *Melee Spell (5ft):* +2 to hit. *On Hit:* 10 (3d6) fire damage.

Saving Throws

A 'saving throw' in the context of the main body of an `RpgStat` is *an action which forces creatures to perform a saving throw*, such as a dragon's breath weapon.

`\RpgStatSaveEffect { name }[<options>]`

`\save`

Typesets an effect called `name` which forces a saving throw, such as a breath weapon, followed by its effects on a failed and a successful save. The `options` are:

Option	Effect	Default
`dc`	The difficulty class of the save.	10
`failure` *also* `fail`	The effect on a failed save.	*(none)*
`modifier`	The ability used for the save: `str`, `dex`, `con`, `int`, `wis` or `cha`, or the same in capitals.	`dex`
`origin`	Where an area effect is centred. **Notes:** Shown as 'Choose a point ...' before the area.	*(none)*
`shape`	`single`, `cone`, `sphere`, `line` or `cube`. **Notes:** `single` affects one target within the `size`; the others affect every target in an area of that shape and `size`.	`single`
`size`	The range, or the size of the area, in feet. **Notes:** The units are added if missing.	60
`success` *also* `succeed`	The effect on a successful save. **Notes:** Left out if it is empty.	*(none)*
`target`	`creature` or `enemy`.	`creature`
`text`	Text placed before the description of the save.	*(none)*

Saving Throws

```
\begin{RpgStat}{Basilisk}[con=15, cr=3]
   \section{Actions}
      \save{Petrifying Gaze}[modifier=con,
        dc=12,
          failure={The target begins to turn
            to stone.}, success={The target is
            unaffected.}]
      \save{Venom Spray}[modifier=dex, dc=12,
        shape=cone, size=15,
          failure={\RpgDice{3d6} poison
            damage}, success={Half damage}]
\end{RpgStat}
```

Basilisk

Medium Creature, Neutral

AC	10	**Speed**	30ft
HP	4 (1d8)	**Initiative**	+0

	STR (10)	DEX (10)	CON (15)	INT (10)	WIS (10)	CHA (10)
MOD	+0	+0	+2	+0	+0	+0

Senses passive Perception 10
Challenge 3 (700xp, proficiency: +2)

Actions

Petrifying Gaze. One creature within 60ft must make a DC 12 Constitution save.

On Failure: The target begins to turn to stone.

On Success: The target is unaffected.

Venom Spray. All creatures within a 15ft cone must make a DC 12 Dexterity save.

On Failure: 10 (3d6) poison damage

On Success: Half damage

Spellcasting

`\begin{ RpgStatSpells } [<options>]`
`spells`

Lists the spells the creature can cast, after a sentence giving its bonus to hit and its spell save DC (the proficiency bonus, plus the modifier of its spellcasting ability, plus any extra `bonus`; and 8 more than this, respectively). The `options` are:

Option	Effect	Default
`bonus`	An extra bonus to hit, and to the save DC.	0
`innate`	A flag, which says the spells are cast 'without requiring components'.	*(off)*
`modifier`	The spellcasting ability: `str`, `dex`, `con`, `int`, `wis` or `cha`, or the same in capitals.	`int`
`no-intro`	A flag, which leaves out the introductory sentence.	*(off)*
`title`	The name of the trait.	`Spellcasting`

Each line of spells is typeset with one of the following commands, where the spells are given as a comma-separated list, and are set in italics.

The following commands function anywhere (even outside an `RpgStat`), so long as it is inside a list environment, such as `itemize`.

`\RpgStatSpellList [<uses>]{ spells }`

Used for the modern 'innate' style of spellcasting which does not track spell slots. The spells are typeset as either:

1. 'At will' (if no <uses> argument given)
2. '<x>/day' (if only one spell given)
3. '<x>/day each' (if more than one spell given)

`\RpgStatSpellSlots [<level>][<slots>]{ spells }`

Used for the traditional slot-based spellcasting. The spells are typeset as either:

1. 'Cantrip' (if no <level> argument given)
2. '`\RpgOrdinal{ <level> }` level Spell (at will)' if a <level> is given, but no <slots>

3. '\RpgOrdinal{ <level> } level Spell (<slots> slots)' if both a <level> and <slots> are given.

Within RpgStatSpells, the following short names are also available. Like the other short names (page 44), each is only created if no command of that name already exists, and they cannot be used outside the environment:

`\atWill { spells }`

Lists spells which can be cast at will. It is the same as \RpgStatSpellList{ spells }.

`\level [<level>][<slots>]{ spells }`

Lists spells which are cast using spell slots. It is the same as \RpgStatSpellSlots[<level>][<slots>]{ spells }.

`\perDay { uses }{ spells }`

Lists spells which can be cast uses times per day. It is the same as \RpgStatSpellList[<uses>]{ spells }, except that the number of uses is required.

Spellcasting

```
\begin{RpgStat}{Mage}[int=17, cr=6]
    \section{Actions}
        \begin{RpgStatSpells}[modifier=int]
            \atWill{mage hand, light}
            \perDay{2}{fly, counterspell}
            \level{fire bolt, shocking grasp}
            \level[1][4]{magic missile, shield}
        \end{RpgStatSpells}
        \begin{RpgStatSpells}[modifier=cha,
          title=Innate Spellcasting, innate]
            \perDay{1}{misty step}
            \level[9]{Wish}
        \end{RpgStatSpells}
\end{RpgStat}
```

Mage

Medium Creature, Neutral

AC	10	**Speed**	30ft
HP	4 (1d8)	**Initiative**	+0

	STR (10)	DEX (10)	CON (10)	INT (17)	WIS (10)	CHA (10)
MOD	+0	+0	+0	+3	+0	+0

Senses passive Perception 10
Challenge 6 (2200xp, proficiency: +3)

Actions

Spellcasting. Mage may cast the following spells. They have +6 to hit, save DC 14

At will: *mage hand, light*
2/day each: *fly, counterspell*
Cantrips: *fire bolt, shocking grasp*
1st level (4 slots): *magic missile, shield*

Innate Spellcasting. Mage may cast the following spells without requiring components. They have +3 to hit, save DC 11

1/day: *misty step*
9th level (at will): *Wish*

Legendary & Mythic Actions

The following commands are used to initiate the Legendary or Mythic actions section of a statblock – in addition to the usual \section command, these insert the explanatory preambles.

`\RpgStatLegendarySection [<options>]`

Begins the 'Legendary Actions' section, with the standard explanation of legendary actions. The actions follow, each begun with \action (with its cost in legendary actions, if more than one). The options are:

Option	Effect	Default
defiance	A flag, which adds the homebrew 'Legendary Defiance' action.	*(off)*
uses	The number of legendary actions the creature may take.	3

`\RpgStatMythicSection` `[<options>]`

Begins the 'Mythic Monster' section: the mythic trait, followed by the traits and legendary actions the creature gains when it is activated. The `options` are:

Option	Effect	Default
`narration`	Text to be read aloud when the mythic trait activates, shown in an `RpgNarration` (page 19).	*(none)*
`passive`	A flag, for a creature which gains only traits, and no legendary actions.	*(off)*
`trait`	The description of the mythic trait. **Notes:** By default, the trait restores the creature to full hit points, and removes any negative effects, when it is reduced to 0 hit points.	*(see notes)*
`trait-name`	The name of the mythic trait.	`Second Phase`

`\RpgStatLegendaryDefiance` `{}`

> **Legendary Defiance is Homebrew**
>
> Legendary Defiance is *not* part of the official D&D rules: it is a homebrew action, written for `RPGTeX` as an improved version of Legendary Resistance.
>
> As a legendary action, the creature repeats a saving throw which it failed in the past hour, against an effect which is still ongoing. Any penalties from that effect are ignored for the new save, and the effect ends on a success.
>
> It is never added to a statblock unless it is asked for, with `\RpgStatLegendaryDefiance` or the `defiance` option of `\RpgStatLegendarySection`. Check that it suits your table before using it.

Adds the homebrew 'Legendary Defiance' action (see the sidebar above). It is usually added with the `defiance` option of `\RpgStatLegendarySection`.

`\RpgStatLegendaryResistance` `{ uses }`

Adds the standard 'Legendary Resistance' trait, which may be used `uses` times per day.

Legendary & Mythic Actions, in a Full-Width Statblock

```
\begin{RpgStat*}{Ancient Wyrm}[size=Gargantuan, type=Dragon, cr=20]
    \section{Traits}
        \RpgStatLegendaryResistance{3}
    \RpgStatLegendarySection[uses=3, defiance]
        \action{Tail Swipe} The wyrm makes a tail attack.
        \action[2]{Wing Attack} The wyrm beats its wings.
    \RpgStatMythicSection[narration={The wyrm's scales blaze with light, and it rises again!}]
        \action{Radiant Breath} The wyrm exhales radiant fire.
\end{RpgStat*}
```

Ancient Wyrm

Gargantuan Dragon, Neutral

AC 10 **Speed** 30ft
HP 4 (1d8) **Initiative** +0

	STR (10)	DEX (10)	CON (10)	INT (10)	WIS (10)	CHA (10)
MOD	+0	+0	+0	+0	+0	+0

Senses passive Perception 10
Challenge 20 (23200xp, proficiency: +6)

Traits

Legendary Resistance (3/day). When Ancient Wyrm fails a saving throw, it can choose to succeed instead.

Legendary Actions (3 uses)

Immediately after another creature's turn, Ancient Wyrm can expend a Legendary Action to take one of the following actions. Expended actions are regained at the start of each of its turns.

Legendary Defiance (1 action). Ancient Wyrm repeats a saving throw that they failed in the past hour which imposed an ongoing effect. Any penalties associated with the chosen effect are ignored whilst making this new saving throw. On a success the effect is ended.

Tail Swipe (1 action). The wyrm makes a tail attack.

Wing Attack (2 actions). The wyrm beats its wings.

Mythic Monster

Second Phase. When reduced to 0HP, Ancient Wyrm recovers all hit points and removes all negative status effects.

When the mythic trait activates, read the following aloud:

> The wyrm's scales blaze with light, and it rises again!

When the mythic trait is activated, Ancient Wyrm gains the following traits and Legendary Actions.

Radiant Breath (1 action). The wyrm exhales radiant fire.

Full-Width Statblocks

`\begin{ RpgStat* } [<card-options>]{ name }[<properties>]`

The same as `RpgStat`, but sets the `twocolumn` property, and places the statblock in a `figure*` float. In a two-column document, this gives a statblock spanning the width of the page, which is useful for large creatures. If no `float` placement is given, the statblock is placed where it appears in the text, using the `strip` environment of the `cuted` package; this is fragile, and may fail near other floats, or with too little text around it.

The example of legendary and mythic actions (page 49) uses `RpgStat*`: since this manual is in one column, the statblock is placed where it appears, with its contents in two columns.

Card Mode

In card mode, which is turned on with `\RpgSwitch{ UseStatCard }{ on }` (page 18), the statblock is typeset on a card, without its frame and background. The abilities, and the commands which create them, are the same.

Legacy Appearance

The appearance of the statblock follows the 2024 rules. The appearance of the 2014 rules is also available:

`\RpgStatLegacyMode {}`

Gives subsequent statblocks the 2014 appearance, until the end of the current group. Only the appearance changes: the statistics are computed in the same way.

The 2014 Appearance

```
{
    \RpgStatLegacyMode{}
    \begin{RpgStat}{Commoner}[type=Humanoid,
      languages=Common]
        \section{Actions}
            \melee{Club}[dmg=1d4,
              dmg-type=bludgeoning]
    \end{RpgStat}
}
% outside the braces, the 2024 appearance
  returns
```

Commoner

Medium Humanoid, Neutral

AC 10
HP 4 (1d8)
Speed 30ft

STR	DEX	CON	INT	WIS	CHA
10 (+0)	10 (+0)	10 (+0)	10 (+0)	10 (+0)	10 (+0)

Senses passive Perception 10
Languages Common
Challenge 0 (0xp, proficiency: +2)

Actions

Club. *Melee (5ft):* +2 to hit. *On Hit:* 2 (1d4) bludgeoning damage.

Customising the dnd Theme

You can ignore this section
This section is for those who wish to adjust the appearance or wording of the dnd theme, rather than to use it.

General changes can be made by using the exposed \RpgTheme... commands documented in Part III; for example modifying the font uses \RpgThemeFont (page 66).

In addition to these general actions, the dnd theme exposes the following for dnd-specific modifications. Since loading the theme defines each of the following, any changes must be made after the theme is loaded, and are undone if it is loaded again with \RpgSetTheme (page 12).

Statblocks

Colors

Every statblock is drawn in the following colors (page 35), which may be redefined to change the appearance of all statblocks:

Color	Used for
StatblockRule	The tapered rules. A pale shade of it is also used as the ThemeColor within the statblock, and so for its tables.
StatblockHeading	The title, the section headings, and the text of the header block (from the armor class to the challenge rating).
StatblockFrame	The frame, and the filigree of filigree-match.
StatblockBackground	The background, unless the background option is off, when it is white.
StatblockRibbon	The borders at the top and bottom, in the legacy appearance (page 51).

Since \colorlet copies a color at the moment it is used, a color defined in terms of another (such as StatblockHeading, which is a dark shade of StatblockRule) does not change when the other is redefined: each must be redefined separately. The color and frame-color properties (page 43) override these colors within a single statblock.

The Frame

Each statblock (outside of card mode) is framed by a tcolorbox, whose appearance is held in the rpgstatbox style. It may be adjusted in the same way as the styles of the other boxes (page 71); for example,

```
\tcbset{rpgstatbox/.append style={sharp corners}}
```

removes the rounded corners of every statblock which follows. \RpgStatLegacyMode (page 51) replaces the style with the legacy appearance, so any changes should be made after it is called.

Rules

The tapered rules which separate the parts of a statblock are drawn by:

`\RpgStatLine {}`

Draws a tapered rule across the width of the line, in StatblockRule. It may also be used outside of a statblock.

Part Pages

`\RpgThemePartPageTikz {}`

Draws the badge at the top of each part page (page 37), which is placed with its top at the top edge of the page, and centred. It may be redefined (for example, with \RenewDocumentCommand) to draw a different badge. \RpgThemeSetDMGParts (page 37) and \RpgThemeSetPHBParts (page 37) work by redefining it, and so replace any custom badge.

Wording

The fixed text of the legendary and mythic sections (page 49) is held in the following commands, which may be redefined with `\renewcommand` (for example, to translate them). Within them, `\RpgStatName` (page 44) gives the name of the creature.

`\rpgstringlegendaryDefiance` *(Variable)*

The description of the homebrew 'Legendary Defiance' action.

`\rpgstringlegendaryResistance` *(Variable)*

The description of the 'Legendary Resistance' trait.

`\rpgstringlegendarySpiel` *(Variable)*

The explanation of legendary actions, which begins the 'Legendary Actions' section.

`\rpgstringmythicDefaultBody` *(Variable)*

The default description of the mythic trait, used if no `trait` is given to `\RpgStatMythicSection` (page 50).

`\rpgstringmythicGains` *(Variable)*

The sentence which introduces the traits a creature gains when its mythic trait activates.

`\rpgstringmythicNarrationLeadIn` *(Variable)*

The sentence which introduces the `narration` of `\RpgStatMythicSection` (page 50).

Chapter 8: scifi Theme

The **scifi** theme gives a document a futuristic appearance, with angular fonts, a cool grey page and a starship-style footer. It begins by loading the default theme (page 28), and so only changes the elements described below.

Appearance

Fonts

If the **font** option is active, the **scifi** theme defines three font families. Unlike those of the **dnd** theme, they are not part of TeX Live: they are distributed with the theme, in **themes/scifi/fonts**, alongside their licences.

`\galaxy {}`

Galaxy Edge, a condensed display font, used for the title, part pages and drop capitals.

`\jura {}`

Jura, a rounded sans-serif font, used for the main body text, and for the text of tables, tips, sidebars, narration and statblocks. Since Jura's own small capitals cover only some letters, its small capitals are borrowed from Fira Sans.

`\scifi {}`

Orbitron, a geometric display font, used for the subtitle, the headings from chapters to subparagraphs, the titles and headers of tables, tips and sidebars, the parts and chapters of the table of contents, the footer and page numbers, and emphasis.

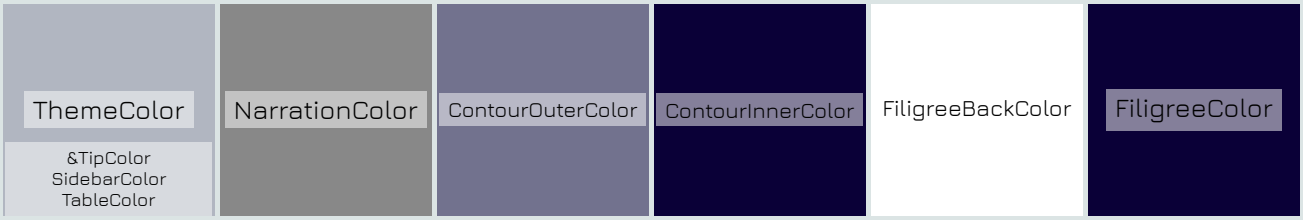
The values assigned to the font elements are as follows (note that **\normalfont** is aliased to the value of \RpgFontBody (page 67)):

Element	FontFamily	FontStyle	Example
\RpgFontBody	\jura		Lorum ipsum
\RpgFontEmphasis	\scifi	\bfseries	**Lorum ipsum**
\RpgFontTitle	\galaxy	\fontsize {80}{50}\selectfont	Lorum ipsum
\RpgFontSubTitle	\scifi	\Large	Lorum ipsum

\RpgFontPart	\normalfont	\fontsize {60}{50}\selectfont \color {TitleBlue}\galaxy \scshape	Lorum ipsum
\RpgFontChapter	\normalfont	\scifi \Huge	Lorum ipsum
\RpgFontSection	\normalfont	\scifi \huge	Lorum ipsum
\RpgFontSubsection	\normalfont	\scifi \Large	Lorum ipsum
\RpgFontSubsubsection	\normalfont	\scifi \large	Lorum ipsum
\RpgFontParagraph	\normalfont	\bfseries \it \scifi	Lorum ipsum
\RpgFontSubparagraph	\normalfont	\bfseries \it \scifi	Lorum ipsum
\RpgFontTableTitle	\scifi	\bfseries \large	Lorum ipsum
\RpgFontTableHeader	\scifi	\bfseries	Lorum ipsum
\RpgFontTableBody	\normalfont	\small	Lorum ipsum
\RpgFontTipTitle	\scifi	\bfseries \scshape	Lorum ipsum
\RpgFontTipBody	\normalfont	\small	Lorum ipsum
\RpgFontSidebarTitle	\scifi	\bfseries \scifi	Lorum ipsum
\RpgFontSidebarBody	\normalfont	\small	Lorum ipsum
\RpgFontNarration	\normalfont	\small	Lorum ipsum
\RpgFontAbstractTitle	\Large	\scshape \bfseries	LORUM IPSUM
\RpgFontAbstractBody	\normalfont	\small \slshape	*Lorum ipsum*
\RpgFontSecretTitle	\normalfont	\bfseries	Lorum ipsum
\RpgFontCardTitle	\normalfont	\bfseries \Large	Lorum ipsum
\RpgFontCardHeader	\normalfont	\it	*Lorum ipsum*
\RpgFontCardBody	\normalfont	\small	Lorum ipsum
\RpgFontTocPart	\normalfont	\Large \scifi	Lorum ipsum
\RpgFontTocChapter	\normalfont	\large \scifi	Lorum ipsum
\RpgFontTocSection	\normalfont	\normalsize	Lorum ipsum
\RpgFontStatBlockTitle	\normalfont	\bfseries \scifi \LARGE	Lorum ipsum
\RpgFontStatBlockBody	\normalfont	\small	Lorum ipsum
\RpgFontStatBlockSection	\scifi	\scifi \large	Lorum ipsum
\RpgFontFooter	\normalfont	\scifi \scriptsize \textcolor {SlateGray}	orum ipsum
\RpgFontPageNumber	\normalfont	\scifi \small \textcolor {SlateGray}	Lorum ipsum
\RpgFontDropCap	\galaxy		Lorum ipsum
\RpgFontDropCapAfter	\normalfont	\slshape	*Lorum ipsum*

Colors

The `scifi` theme assigns the following values to the standard colors:



PaperColor

SecretColor

It also defines a number of colors of its own:

SlateBlue — theme color

SlateGray — paper & footer text

TitleBlue — titles

MidBlue — contour

MidGray — narration

Backgrounds & Footers

Each page is given a plain background in `SlateGray`, the `PaperColor` of the theme. The footer is a dark blue panel across the bottom of the page, which is mirrored on alternate pages in two-sided documents. The name of the chapter is placed on the panel, in `\RpgFontFooter` (page 68), and the page number sits in a notch at its outer corner, in `\RpgFontPageNumber` (page 68); both are set in `SlateGray`.

Text Boxes

The `scifi` theme gives the sidebar the following appearance; the tip and narration are the same as in the default theme (page 28), but use the fonts of the `scifi` theme:

The RpgSidebar

Red 'hazard stripes' fade across the top and bottom.

The RpgTip

Almost entirely undecorated.

RpgNarration uses the narration color.

Filigree

The `scifi` filigree frame (page 21) resembles a targeting reticle, after the ruled boxes of the *Traveller* rulebooks: a chamfered bracket sits just outside each corner, hairlines run along the top and bottom, and three slanted stripes, echoing those of the sidebar, cross the hairlines beside the top-left and bottom-right corners. It is drawn in `FiligreeColor`, which the theme sets to `TitleBlue`, and may be applied to any `tcolorbox`, including tables:

Filigree

```
\begin{RpgFiligree}
    Some decorative text
\end{RpgFiligree}

\vspace{1cm}

\begin{RpgTable}[filigree]{lX}
    Header 1 & Header 2
    \\
    Body 1 & Body 2
    \\
    Body 3 & Body 4
\end{RpgTable}
```

Some decorative text

Header 1	Header 2
Body 1	Body 2
Body 3	Body 4

Title & Part Pages

The compact title is the same as in the default theme (page 29), but uses the fonts of the `scifi` theme:

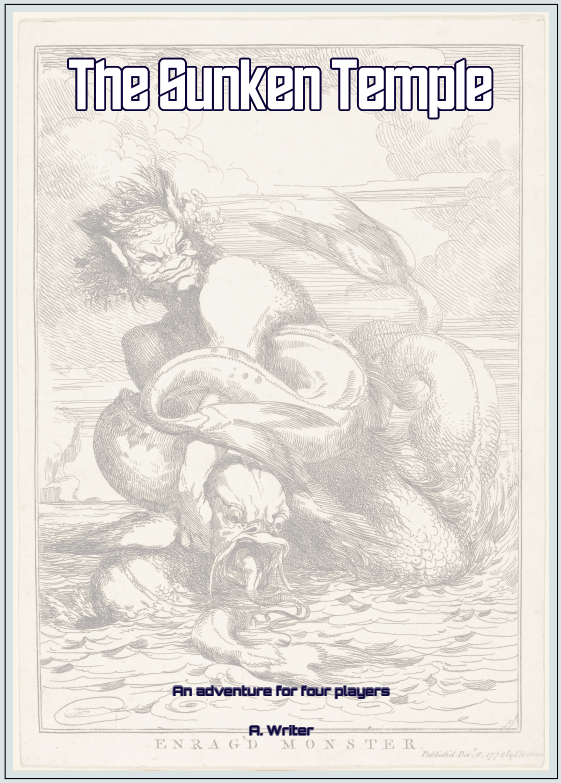
A Compact Title

```
\RpgThemeTitleMode{header}
\title{The Sunken Temple}
\subtitle{An adventure for four players}
\author{A. Writer}
\RpgMakeTitle
```

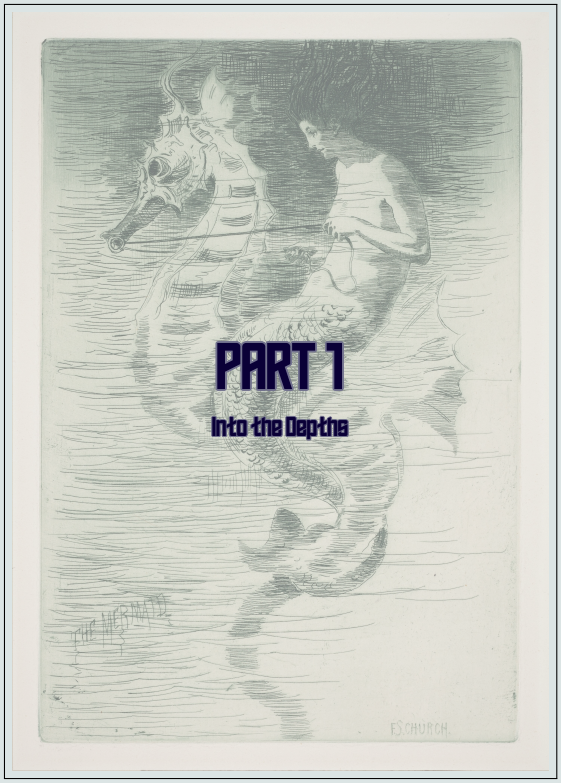
On the cover page, the title is set in white, outlined in `TitleBlue`, at the top of the page, and shrunk if necessary to fit its width. The subtitle and author are placed at the bottom of the page, outlined with `\RpgContour` (page 15) in `ContourInnerColor` and `ContourOuterColor`. If these two colors are made the same, the subtitle and author are instead set in that color, without an outline (`\RpgIfColoursEqualTF` (page 62)).

On a part page, the word 'Part' and the part's number are placed in the centre of the page, in capitals, with the name of the part beneath them; both are set in `TitleBlue`, outlined in a paler blue. Parts are numbered with Arabic numerals, rather than Roman ones.

The cover and part pages of a short example document, typeset with the `scifi` theme, are:



Cover page



Part page

Section Headers

As in the default theme, only parts and chapters are numbered. Headings are set in `\scifi`:

Section Headers

```
\section*{Example} %starred so not added to
  toc; otherwise the same
    \subsection{Another Example}
        \subsubsection{More Examples!}
            \paragraph{A Paragraph} Its text
              runs on from the heading.

            \subparagraph{A Subparagraph} As
              does this one.
```

Example

Another Example

More Examples!

A Paragraph. Its text runs on from the heading.
A Subparagraph. As does this one.

Table of Contents

The table of contents is laid out as in the default theme (page 28), except for the entries for parts, which are centred between two rules, with 'Part' and the part's number above the name of the part.

RpgDice

The format of `\RpgDice` (page 15) is the same as in the default theme.

Rule Environments

The `scifi` theme does not configure any of the rule environments (page 25). Items (page 31), feats (page 32), spells and statblocks (page 32) have the same properties and layout as in the default theme, but use the fonts and colors of the `scifi` theme.

An item, in text mode and in card mode:

RpgItem: Text Mode

```
\begin{RpgItem}{Mag-Anchor}[
    description={Hull-Grip Boarding Clamp},
    image={img/Phaser_model.jpg}
]
    A magnetic clamp, standard issue for
      boarding crews. While it is fixed to a
      metal surface, you cannot be pushed,
      pulled or thrown, even in zero gravity.
      Detaching it takes an action.
\end{RpgItem}
```

Mag-Anchor
Hull-Grip Boarding Clamp

A magnetic clamp, standard issue for boarding crews. While it is fixed to a metal surface, you cannot be pushed, pulled or thrown, even in zero gravity. Detaching it takes an action.

RpgItem: Card Mode

```
\RpgSwitch{UseItemCard}{on}
\begin{RpgItem}[width=5cm,
  height=7cm]{Mag-Anchor}[
    description={Hull-Grip Boarding Clamp},
    image={img/Phaser_model.jpg}
]
    A magnetic clamp, standard issue for
      boarding crews. While it is fixed to a
      metal surface, you cannot be pushed,
      pulled or thrown, even in zero gravity.
      Detaching it takes an action.
\end{RpgItem}
\RpgSwitch{UseItemCard}{off}
```

Mag-Anchor
Hull-Grip Boarding Clamp

A magnetic clamp, standard issue for boarding crews. While it is fixed to a metal surface, you cannot be pushed, pulled or thrown, even in zero gravity. Detaching it takes an action.

A feat, in text mode and in card mode:

RpgFeat: Text Mode

```
\begin{RpgFeat}{Void Dancer}[requires={EVA
  certification}]
    Years of spacewalks have taught you to move
      without gravity. You suffer no penalties
      for fighting in zero gravity, and once
      per scene you may push off any surface to
      move \RpgDice{2d6} metres as a free
      action.
\end{RpgFeat}
```

Void Dancer

Requires: EVA certification

Years of spacewalks have taught you to move without gravity. You suffer no penalties for fighting in zero gravity, and once per scene you may push off any surface to move 2d6 metres as a free action.

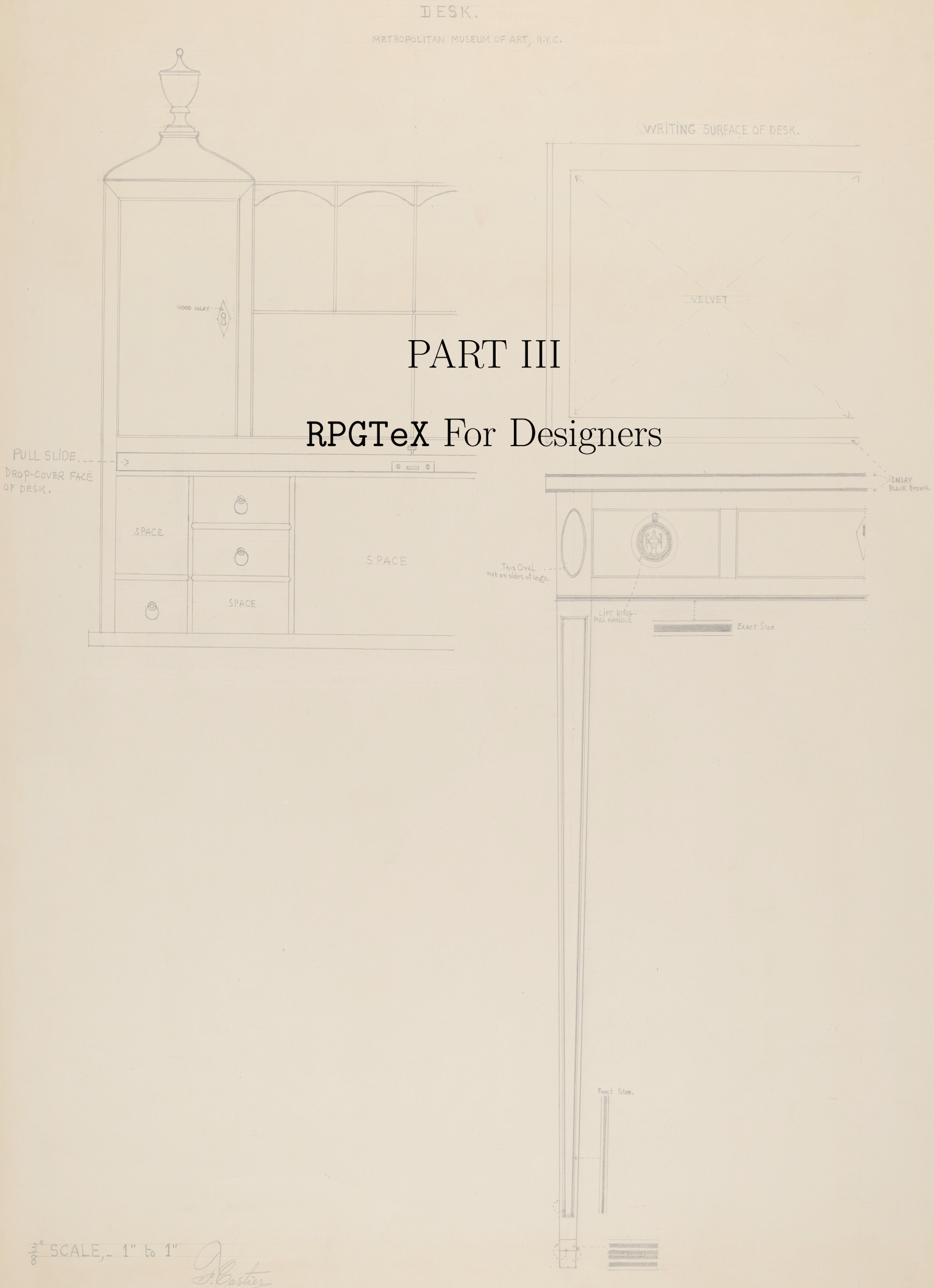
RpgFeat: Card Mode

```
\RpgSwitch{UseFeatCard}{on}
\begin{RpgFeat}[width=5cm, height=6cm]{Void
  Dancer}[requires={EVA certification}]
    Years of spacewalks have taught you to move
      without gravity. You suffer no penalties
      for fighting in zero gravity, and once
      per scene you may push off any surface to
      move \RpgDice{2d6} metres as a free
      action.
\end{RpgFeat}
\RpgSwitch{UseFeatCard}{off}
```

Void Dancer

Years of spacewalks have taught you to move without gravity. You suffer no penalties for fighting in zero gravity, and once per scene you may push off any surface to move 2d6 metres as a free action.

Requires: EVA certification



PART III

RPGTeX For Designers

Chapter 9: Building a Theme

This part of the manual documents the commands used to build themes. They are intended for designers creating or modifying a theme, and for users who wish to adjust an existing theme in detail. By convention, all of these commands begin with `\RpgTheme...`.

Authors of RPGTeX documents probably do not need any of the commands in this part, and it can safely be ignored.

This chapter describes how a theme is organised. The chapters which follow describe how a theme defines its fonts (page 66), the design of its pages (page 69), the appearance of individual elements (page 71), and its rule environments (page 75).

Code Setters

Many of the commands in this part of the manual do not perform any action themselves. Instead, they ***store*** a piece of code, which RPGTeX runs later, when the corresponding element is created. For example, `\RpgThemePartPage` stores the code used to draw a part page, which is run each time `\part` is called.

The stored code is often given arguments at the time it is run, such as the name of the part. These are accessed as `#1`, `#2` and so on within the stored code, even though the setter itself is called with only a single argument:

```
\RpgThemePartPage{The part is called #1}
```

The entry for each setter lists the arguments which are available.

Theme Files

When `\RpgSetTheme{ <name> }` is called, (page 12) RPGTeX attempts to input a file of the form:

```
<theme path> / <name> / <name>.cfg
```

Where `<theme path>` is the value set by the theme path option (page 9) or by `\RpgSetThemePath` (page 12). The theme "`name`" is therefore defined by a directory, `<theme path>/name`, containing the file `name.cfg`. This file (the 'theme configuration file') may then execute any LaTeX code.

If the theme config file separates out the concerns across multiple files, it is important to `\input` them using the full file path to prevent name collisions. For example, the default theme's configuration file begins:

```
\input{\RpgPackagePath/themes/default/colors.theme.tex}
\input{\RpgPackagePath/themes/default/utility.theme.tex}
...
```

Theme Code

Since themes are inserted entirely through `\input` commands, and may be introduced midway through a document, there are some restrictions and limitations on what should go inside a theme file or its subsidiary files:

1. **Packages.** Any package loading content (`\usepackage`, `\tcbuselibrary`) should be wrapped in a `\RpgIfPreamble` (page 63) block, and appropriate guards put in place in case the packages cannot be loaded (because the theme was first activated in the document body).
2. **Other preamble-only commands.** Commands such as `\geometry` may also only be used in the preamble. Where a body equivalent exists, `\RpgIfPreambleTF` (page 63) can choose between the two (for example, `\geometry` in the preamble and `\newgeometry` in the body).
3. **Repeated loading.** A theme may be loaded more than once in the same document, either because the author switches back to it, or because another theme builds upon it: the `dnd` and `scifi` themes both begin by calling `\RpgSetTheme{default}`, and so the default theme is reloaded whenever either is activated. Every definition must therefore succeed when it is made a second time:

- Use `\DeclareDocumentCommand` (or `\cs_set:Npn`) rather than `\NewDocumentCommand` (or `\cs_new:Npn`), so that the definition is replaced rather than raising an error. This also allows a later theme to override a command defined by an earlier one.
- Create variables only if they do not already exist (for example, with `\tl_if_exist:NF`), and use `\msg_set:nnn` rather than `\msg_new:nnn`.

4. **Category codes.** A theme file is read with the category codes in force where `\RpgSetTheme` is called. When it is called in the document body, LaTeX3 syntax is off and `@` is not a letter. Each file should therefore begin with `\ExplSyntaxOn` if it uses LaTeX3 syntax, and with `\makeatletter` if it uses internal commands such as `\@title`, rather than relying on the settings left by the file before it.

5. **Timing.** A theme only affects material created after it is loaded. In particular, it cannot change the options passed to the underlying class (such as the font size or the number of columns), and it does not restyle a table of contents which has already been typeset. Changes to the formatting of the contents (with `\titlecontents`) should be wrapped in `\RpgIfPreamble` (page 63): in the document body, the `titletoc` package writes them into the `.toc` file, where they are read on the next compilation before the theme has been loaded, and so may refer to colors or fonts which do not yet exist.

Theme Resources

It is possible to ship non-TeX resources, such as fonts and images, with a theme. By convention, these are kept in subdirectories of the theme's directory, such as `fonts/` and `img/`.

TeX, `graphicx` and `fontspec` search for these files relative to the document being compiled, not to the theme, and so each resource must be referred to by its full path. For example, the `dnd` theme sets its paper texture with:

```
\RpgThemePaperImage{\RpgPackagePath/themes/dnd/img/paper}
```

Note that this uses `\RpgPackagePath` because the theme is bundled with a package; for a general theme which is stored elsewhere, the designer must give the full path.

Fonts are loaded by giving `fontspec` the location of the font files. For example, the `scifi` theme loads its body font with:

```
\RpgIfFont{
  \providefontfamily{\jura}{Jura}[
    Path = \RpgPackagePath/themes/scifi/fonts/,
    UprightFont = *-Regular,
    ...
  ]
}
```

`\RpgThemeFont` (page 66) may be used unguarded, since it does not load any fonts itself, but the font declarations should be placed within `\RpgIfFont` (page 63), so that fonts are only loaded when the `font` option is active.

Conditional Code

Many parts of a theme are only meaningful when the corresponding package option is active, and some commands may only be used in the preamble, although a theme may be loaded in the document body. A design may also depend on the theme's colors, or on whether the document is two-sided. The following commands allow a theme to include code conditionally:

`\RpgIfColoursEqualTF { color 1 }{ color 2 }{ true code }{ false code }`

`\RpgIfColoursEqual`

Runs `true code` if the two named colors have the same value, and `false code` otherwise. The colors are compared by their RGB values, so colors which are defined in different ways (for example, one with `\definecolor` and the other with `\colorlet`) are equal if they produce the same color. The alias `\RpgIfColoursEqual{ color 1 }{ color 2 }{ code }` takes a single code argument, and is equivalent to `\RpgIfColoursEqualTF{ color 1 }{ color 2 }{ code }{}`.

For example, the `scifi` theme outlines the subtitle on its cover page with `\RpgContour` (page 15), unless `ContourInnerColor` and `ContourOuterColor` are the same, when an outline would be invisible.

`\RpgIfFontTF { true code }{ false code }`

`\RpgIfFont`

Runs `true code` if the `font` option is active (page 8), and `false code` otherwise. The alias `\RpgIfFont{ code }` takes a single argument, and is equivalent to `\RpgIfFontTF{ code }{}`.

`\RpgIfPaperTF { true code }{ false code }`

`\RpgIfPaper`

Runs `true code` if the `background` option is active, and `false code` otherwise. The alias `\RpgIfPaper{ code }` takes a single argument, and is equivalent to `\RpgIfPaperTF{ code }{}`.

`\RpgIfPreambleTF { true code }{ false code }`

`\RpgIfPreamble`

Runs `true code` if it is reached before `\begin{document}`, and `false code` otherwise. The alias `\RpgIfPreamble{ code }` takes a single argument, and is equivalent to `\RpgIfPreambleTF{ code }{}`.

This allows a body alternative to be given for preamble-only commands, for example:

```
\RpgIfPreambleTF{\geometry{...}}{\newgeometry{...}}
```

The restrictions on what a theme may contain are described on page 61.

`\RpgIfSectionsTF { true code }{ false code }`

`\RpgIfSections`

Runs `true code` if the `sections` option is active, and `false code` otherwise. The alias `\RpgIfSections{ code }` takes a single argument, and is equivalent to `\RpgIfSectionsTF{ code }{}`.

`\RpgIfTitleTF { true code }{ false code }`

`\RpgIfTitle`

Runs `true code` if the `title` option is active, and `false code` otherwise. The alias `\RpgIfTitle{ code }` takes a single argument, and is equivalent to `\RpgIfTitleTF{ code }{}`.

`\RpgIfTwoSideTF { true code }{ false code }`

`\RpgIfTwoSide`

Runs `true code` if the document is two-sided, so that even and odd pages differ, and `false code` otherwise. The alias `\RpgIfTwoSide{ code }` takes a single argument, and is equivalent to `\RpgIfTwoSideTF{ code }{}`.

The provided themes use it to set the footer of even pages only in two-sided documents, since `fancyhdr` warns about even-page settings in one-sided ones.

Key-Value Options

Themes often need to give their own commands and environments key-value options. The l3keys library provides a convenient interface, but can be extremely verbose, and relies on excessive boilerplate to declare a new variable. `RPGTeX` provides commands which define such options in a single line each, creating the variable in which the value is stored if it does not already exist. Each key belongs to a `family`, such as `rpg/table`.

Defining Keys

Keys may be defined one at a time, giving the `family` each time (page 63), or several at once within a single `family` (page 64). The two sets of commands define exactly the same kinds of key.

Defining Individual Keys

Each of the following commands defines a single key in the given `family`:

`\RpgThemeNewBoolKey (*){ family }{ key }{ variable }`

Defines a flag, which takes no value. The unstarred form sets the boolean `variable` to false, and makes it true when the flag is present. The starred form does the opposite: the variable starts true, and the flag makes it false.

`\RpgThemeNewCodeMapKey { family }{ key }{ choices }{ code }{ default }`

Defines a key whose value must be one of the comma-separated `choices`. Rather than storing the value, it runs the corresponding entry of the comma-separated `code` list. Any entry of `code` which itself contains a comma must be enclosed in braces.

`\RpgThemeNewTLChoiceKey { family }{ key }{ variable }{ choices }{ default }`

Defines a key whose value must be one of the comma-separated `choices`, and stores it in `variable`.

`\RpgThemeNewTLKey { family }{ key }{ variable }{ default }`

Defines a key which takes any value, and stores it in `variable`. A value must be given whenever the key is used.

`\RpgThemeNewTLMapKey { family }{ key }{ variable }{ choices }{ values }{ default }`

Defines a key whose value must be one of the comma-separated `choices`, and stores the corresponding entry of the comma-separated `values` in `variable`. For example, the `columns` option maps the choices `1,2` to the values `onecolumn,twocolumn`.

Defining Groups of Keys

When several keys belong to the same `family`, they may be defined together, without repeating the family each time:

`\RpgThemeKeys { family }{ definitions }`

Defines several keys in the same `family` at once. The `definitions` are made with the commands below.

```
\RpgThemeKeys{rpg/my-env}
{
    \RpgThemeTLKey {title} {\l_myenv_title_tl} {Untitled}
    \RpgThemeBoolKey {hidden} {\l_myenv_hidden_bool}
    \RpgThemeTLChoiceKey {size} {\l_myenv_size_tl} {small,large} {small}
}

\keys_set:nn {rpg/my-env} {title=Hello, hidden, size=large}
```

The following commands may only be used within the `definitions` of `\RpgThemeKeys`, and define a key in its `family`:

`\RpgThemeBoolKey (*){ key }{ variable }`

Defines a flag, which takes no value. The unstarred form sets the boolean `variable` to false, and makes it true when the flag is present. The starred form does the opposite: the variable starts true, and the flag makes it false. The `family` is the one given to `\RpgThemeKeys`.

`\RpgThemeCodeMapKey { key }{ choices }{ code }{ default }`

Defines a key whose value must be one of the comma-separated `choices`. Rather than storing the value, it runs the corresponding entry of the comma-separated `code` list. Any entry of `code` which itself contains a comma must be enclosed in braces. The `family` is the one given to `\RpgThemeKeys`.

`\RpgThemeTLChoiceKey { key }{ variable }{ choices }{ default }`

Defines a key whose value must be one of the comma-separated `choices`, and stores it in `variable`. The `family` is the one given to `\RpgThemeKeys`.

`\RpgThemeTLKey { key }{ variable }{ default }`

Defines a key which takes any value, and stores it in `variable`. A value must be given whenever the key is used. The `family` is the one given to `\RpgThemeKeys`.

`\RpgThemeTLMapKey { key }{ variable }{ choices }{ values }{ default }`

Defines a key whose value must be one of the comma-separated `choices`, and stores the corresponding entry of the comma-separated `values` in `variable`. For example, the `columns` option maps the choices `1,2` to the values `onecolumn,twocolumn`. The `family` is the one given to `\RpgThemeKeys`.

Setting Keys

Once a key has been defined, its value is set using either the l3keys function `\keys_set:nn`{ family }{ key-value-list } (which requires `\ExplSyntaxOn`), or the LaTeX kernel command `\SetKeys`[<family>]{ key-value-list }, which may be used anywhere. The `family` is optional for `\SetKeys`, but should always be given: if it is omitted, the family of the package currently being loaded is used instead.

The new values are local to the group in which they are set (except where the code run by a `\RpgThemeNewCodeMapKey` key makes a global change), so they revert at the end of the group. Every environment is a group, and a group may also be created with braces, `\begingroup` and `\endgroup`, or `\group_begin:` and `\group_end:`.

Chapter 10: Fonts

RPGTeX allows a high degree of customisation of the fonts and typefaces used throughout a document. Rather than setting fonts directly, each part of the document (headings, table text, footers and so on) draws its font from a named ***font element***, such as \RpgFontSection (page 67). The theme defines the appearance of each element using \RpgThemeFont, and assigns each element to the relevant part of the document. Changing the appearance of an element therefore changes it consistently wherever that element is used.

Why didn't my font change?

\RpgThemeFont does not change any fonts directly: it updates the stored definition of a ***font element***, which a theme designer must then assign to the relevant part of the document. For example, the \RpgFontSection (page 67) element does not 'hook in' to the \section command by itself; it changes the section font only because most themes call \titleformat{\section}{\RpgFontSection}.... If the titleformat were later changed (for whatever reason), the section font would no longer update.

If an element does not change font after its definition is updated, first check that the font element is actually being invoked – and if not, invoke it manually. Once the command is in place, the font will change as expected.

Note also that \RpgThemeFont has no effect unless the font option is active (page 8). This is the default when using one of the RPGTeX classes.

Font Commands

`\RpgThemeFont { key-value-list }`

Updates the font elements specified by a comma-separated list of key-value pairs. Each element has two keys, <name>-family and <name>-style, where <name> is given in the 'Components' column of the table below (or was assigned by \RpgThemeCreateFont).

The values may be any LaTeX font or formatting commands (such as \slshape\scriptsize\bfseries), which are applied whenever the element is invoked. To change the typeface itself, a font family must first be defined, for example using \newfontfamily from the fontspec package.

Font Interface

```
\subsection{The Original Font}
    Here is some text

    \newfontfamily{\courier}{Courier}
    \RpgThemeFont{main-body-family=\courier,
        subsection-style=\slshape\Huge}

\subsection{The New Font}
    And after the change is introduced
```

The Original Font

Here is some text

The New Font

And after the change is introduced

`\RpgThemeCreateFont { \ElementName }{ key-base }`

Creates a new font element, \ElementName, along with the keys <key-base>-family and <key-base>-style. A new element is initially blank (it applies no formatting) until it is assigned a value using \RpgThemeFont. Thereafter, it may be used and modified in exactly the same way as the built-in font elements. Creating an element which already exists (for example, when a theme is loaded again) redefines it, and resets it to blank.

Create Font

```
\RpgThemeCreateFont{\MyFont}{myfont}
\newfontfamily{\courier}{Courier}
\RpgThemeFont{
    myfont-family = \courier,
    myfont-style = \slshape
}


This is text

{\MyFont This is text in my new font}
```

This is text
This is text in my new font

Font Elements

RPGTeX provides 35 font elements by default (themes may define more). Since each element is assigned to part of the document by the theme designer, an element intended for one purpose (such as section headings) may be used for a different purpose within another theme. The descriptions below therefore reflect how the elements are used by RPGTeX itself and by the provided themes.

Family vs Style

Each font element is defined by two components: a `family` and a `style`. Formally, the `family` defines the **typeface** used by the element, whilst the `style` determines the options applied to that typeface (weight, shape, size, colour and so on).

In practice the distinction is largely one of clarity, since the font element simply applies the two in sequence:

```
\RpgFontX = <x-family> <x-style>
```

It is therefore generally safe to place commands which belong in the family within the style key, provided that they do not conflict with other styling.

Font vs Implementation

We encourage designers to place all text styling within the relevant font element, rather than elsewhere. If all subsections are to be red, then set `subsection-style=\color{ red }`, rather than setting the colour within the `titlesec` specification.

Font Element	Components	Usage
`\RpgFontBody`	`main-body-family` `main-body-style`	The main body text of the document. `\RpgThemeFont` sets `\normalfont` equal to this element. Changing the font size here (e.g. with `\large`) can give counterintuitive results, since it affects *only* the body text and does not scale the other elements accordingly. The font size of the whole document should instead be set using the `size` option (page 8).
`\RpgFontEmphasis`	`emph-family` `emph-style`	The font used by `\RpgKey`, and therefore by `\emph` unless the `noemph` option is passed.
`\RpgFontTitle`	`title-family` `title-style`	The font used for the document title, as returned by `\RpgGetTitle` and displayed by `\maketitle`.
`\RpgFontSubTitle`	`subtitle-family` `subtitle-style`	The font used for the subtitle and author, as returned by `\RpgGetSubtitle` and `\RpgGetAuthor` and displayed by `\maketitle`.
`\RpgFontPart`	`part-family` `part-style`	The font used for the part name on the part page created by `\part`.
`\RpgFontChapter`	`chapter-family` `chapter-style`	The font used for `\chapter` headings.
`\RpgFontSection`	`section-family` `section-style`	The font used for `\section` headings.
`\RpgFontSubsection`	`subsection-family` `subsection-style`	The font used for `\subsection` headings.
`\RpgFontSubsubsection`	`subsubsection-family` `subsubsection-style`	The font used for `\subsubsection` headings.

\RpgFontParagraph	paragraph-family paragraph-style	The font used for \paragraph headings.
\RpgFontSubparagraph	subparagraph-family subparagraph-style	The font used for \subparagraph headings.
\RpgFontTableTitle	table-title-family table-title-style	The title of an RpgTable (page 20), if one is set using the title option.
\RpgFontTableHeader	table-header-family table-header-style	The first (header) row of an RpgTable (page 20).
\RpgFontTableBody	table-body-family table-body-style	The rows of an RpgTable (page 20) after the header row.
\RpgFontTipTitle	tip-title-family tip-title-style	The title of an RpgTip (page 19) environment.
\RpgFontTipBody	tip-body-family tip-body-style	The body of an RpgTip (page 19) environment.
\RpgFontSidebarTitle	sidebar-title-family sidebar-title-style	The title of an RpgSidebar (page 19) environment.
\RpgFontSidebarBody	sidebar-body-family sidebar-body-style	The body of an RpgSidebar (page 19) environment.
\RpgFontNarration	narration-family narration-style	All text within an RpgNarration (page 19) environment (which has no title).
\RpgFontAbstractTitle	abstract-title-family abstract-title-style	The title of an RpgAbstract (page 21) environment.
\RpgFontAbstractBody	abstract-body-family abstract-body-style	The body of an RpgAbstract (page 21) environment.
\RpgFontSecretTitle	secret-title-family secret-title-style	The labels which distinguish the hidden and public text of an RpgSecret (page 22) environment when secrets are shown.
\RpgFontCardTitle	card-title-family card-title-style	The title of a card, such as a rules-environment in card-mode (such as RpgItem).
\RpgFontCardHeader	card-header-family card-header-style	Subheadings and descriptive headers within a card (such as the type and rarity of an RpgItem).
\RpgFontCardBody	card-body-family card-body-style	The body text of an RpgCard (page 24).
\RpgFontTocPart	toc-part-family toc-part-style	Part entries in the table of contents.
\RpgFontTocChapter	toc-chapter-family toc-chapter-style	Chapter entries in the table of contents.
\RpgFontTocSection	toc-section-family toc-section-style	Section entries in the table of contents.
\RpgFontStatBlockTitle	stat-block-title-family stat-block-title-style	An optional font which is applied to RpgStat environments by some themes to render the title.
\RpgFontStatBlockBody	stat-block-body-family stat-block-body-style	An optional font which is applied to RpgStat environments by some themes to render the main body text.
\RpgFontStatBlockSection	stat-block-section-family stat-block-section-style	An optional font which is applied to RpgStat environments by some themes to render the \section commands.
\RpgFontFooter	footer-family footer-style	The footer text.
\RpgFontPageNumber	page-number-family page-number-style	The page number within the footer.
\RpgFontDropCap	drop-cap-family drop-cap-style	The large initial letter created by \RpgDropCap.
\RpgFontDropCapAfter	drop-cap-after-family drop-cap-after-style	The text which follows the initial letter of an \RpgDropCap, on the first line.

Chapter 11: Page Design

This chapter describes the commands which define the overall design of a theme's pages: their backgrounds and footers, and the title and part pages.

Backgrounds & Footers

These commands control the page background and footer. Their effects depend on the `bg` option (page 8): the paper colour and image are only shown when it is `full`, and the footer decorations unless it is `none`. The paper and footer commands have no effect unless the `background` option is active.

The footer commands draw on the page using the `eso-pic` package. Positions are measured from the bottom-left corner of the page, and further content may be placed with `\RpgPut` (page 17).

`\RpgThemeFooterDecoration` `(*){ height }{ path/to/image }{ overlay }`

Draws an image across the full width of the bottom of the current page, with the given `height` (which may be "!" to use the aspect-ratio height), and then draws the `overlay` (such as `\RpgPut` or `\RpgThemePageNumberAt` commands) on top of it.

The decoration is only drawn on the page on which the command is called, and so it is normally placed within the definition of a page footer. The starred form mirrors the image horizontally, and the unstarred form draws it as it is, allowing a single image to be used on both left- and right-hand pages:

```
\fancyfoot[LE]{%
    \RpgThemeFooterDecoration{1.5cm}{path/to/scroll}{
        \RpgPut{0.43in}{0.15in}{\RpgFontPageNumber \thepage}
    }
}
\fancyfoot[RO]{%
    \RpgThemeFooterDecoration*{1.5cm}{path/to/scroll}{
        \RpgPut{\paperwidth-0.43in}{0.15in}{\RpgFontPageNumber \thepage}
    }
}
```

`\RpgThemePageNumberAt` `{ x }{ y }`

Typesets the page number, in the `\RpgFontPageNumber` (page 68) font element, at the position (`x`, `y`), with the alignment set to `b` (centred, with the baseline at height `y`). It may only be used within a page overlay, such as the final argument of `\RpgThemeFooterDecoration`.

`\RpgThemePaperColor` `{ colour }`

Sets the background colour of every page, and updates `PaperColor` to match (page 10).

`\RpgThemePaperImage` `{ path/to/image }`

Sets an image, stretched to fill the page, as the background of every subsequent page. This is typically a paper texture. An empty path removes any existing background image.

Title & Part Pages

These commands define the appearance of the pages created by `\maketitle` (page 12) and `\part` (page 13). The cover and part page designs are drawn on the page with `eso-pic`, usually with `\RpgPut` (page 17), while the header title is typeset as ordinary text. Within them, the title information is available through `\RpgGetTitle`, `\RpgGetSubtitle` and `\RpgGetAuthor` (page 12).

`\RpgThemeOnPartChange { code }`

Stores `code` to be run after every `\part` (in either form). For example, the `dnd` theme uses this to cycle its colour scheme with each new part.

`\RpgThemePartPage { code }`

Stores the design of the part page created by `\part`, which is drawn on top of the part's background image (if any). The available arguments are:

Argument	Value
`#1`	The name of the part.

The part number is available through `\thepart`. By default, the part number and name are centred on the page in the `\RpgFontPart` (page 67) font element.

`\RpgThemeTitleCover { code }`

Stores the design of the cover page created by `\maketitle` in `cover` mode. It is drawn on top of the `\cover` image, if one has been set, or on an otherwise blank page.

`\RpgThemeTitleHeader { code }`

Stores the code which typesets the title when `\maketitle` is called in `header` mode. Unlike the cover, this is ordinary text, placed at the point where `\maketitle` is called.

`\RpgThemeTitleMode { cover / header }`

Selects which of the two designs `\maketitle` produces. Any other value raises an error. The `rpgbook` class selects `cover`; otherwise, the default is `header`.

Chapter 12: Styling Elements

Box Formatters

The appearance of most environments is controlled through `tcolorbox` styles, which a theme may redefine with `\tcbset`:

```
\tcbset{rpgsidebar/.style={colframe=black, ...}}
```

The available styles are:

Style	Used by
`rpgcard`	The `RpgCard` (page 24) environment.
`rpgnarration`	The `RpgNarration` (page 19) callout.
`rpgsecret`	The box used by `RpgSecret` (page 22).
`rpgsidebar`	The `RpgSidebar` (page 19) callout.
`rpgtip`	The `RpgTip` (page 19) callout.

Redefining a style with `/.style` replaces it entirely, including the theme's settings. To adjust a style instead, add to it with `/.append style`:

Restyling a Callout

```
\tcbset{
    rpgtip/.append style={
        sharp corners,
        borderline west={3pt}{0pt}{black},
    }
}

\begin{RpgTip}{A Restyled Tip}
    This tip has square corners, and a rule
      down its left-hand side.
\end{RpgTip}
```

A Restyled Tip

This tip has square corners, and a rule down its left-hand side.

Other Formatters

The following commands control the elements which are not built from a `tcolorbox` style.

`\RpgThemeDiceFormat { code }`

Stores the code which typesets a dice expression, after it has been parsed by `\RpgDice` (page 15). The available arguments are:

Argument	Value
`#1`	The number of dice (empty if none was given).
`#2`	The number of sides.
`#3`	The evaluated modifier, including its sign (for example, `+6` or `-2`).

By default, the expression is typeset as `#1d#2#3`.

The arguments are passed as variables which contain these values, rather than as the values themselves. They typeset correctly as they are, but must be expanded before being tested or compared: for example, with `\tl_if_blank:eTF` rather than `\tl_if_blank:nTF`.

RpgDice Formatting: The D&D Format

```
Before: \RpgDice{2d8 + 3}, \RpgDice{d8}

\ExplSyntaxOn
\RpgThemeDiceFormat{ % #1: Dice Number,
  #2: Dice Sides, #3: Modifier
    %% d8 -> 1d8
    %% the arguments are variables, so
      must be expanded before testing
    \tl_set:Ne \l_tmpa_tl {
      \tl_if_blank:eTF {#1} {1} {#1} }
    %% compute the average roll, and
      print it first
    \fp_eval:n { floor ( \l_tmpa_tl * (
      #2 + 1 ) / 2 ) + ( #3 ) }
    ~( \l_tmpa_tl d#2 \fp_compare:nNnF {
      #3 } = { 0 } { #3 } )
}
\ExplSyntaxOff

After: \RpgDice{2d8 + 3}, \RpgDice{d8}
```

Before: 2d8+3, d8
After: 12(2d8+3), 4(1d8)

`\RpgThemeDropCapHeight { lines }`

Sets the number of lines spanned by the initial letter of `\RpgDropCap` (page 16). The default is `4`.

Changing the Drop Cap Height

```
\RpgThemeDropCapHeight{2}

\RpgDropCap{T}{he road} winds down from
  the pass into a valley of grey stone,
  where the river has long since run dry
  and the old watchtowers stand empty.
```

The road winds down from the pass into a valley of grey stone, where the river has long since run dry and the old watchtowers stand empty.

`\RpgThemeMapFormat { code }`

Stores the code which formats one part of the number of an area within an `RpgMap` (page 23). The number of an area is built by running this code once for each level of nesting, from the outermost map inwards, and joining the results. The available arguments are:

Argument	Value
`#1`	The depth being formatted (`1` for the outermost map).
`#2`	The position of the area at that depth (`1` for the first area).

For example, the third area inside the second area of a map is numbered by running the code with `{1}{2}`, and then with `{2}{3}`. By default, the depths cycle through numbers, capital letters, and lower-case Roman numerals preceded by a hyphen, giving numbers such as `1B-iii`.

The code is run during full expansion, and so must be expandable: functions such as `\int_to_Roman:n` may be used, but assignments may not.

RpgThemeMapFormat: Roman Numerals

```
\ExplSyntaxOn
\RpgThemeMapFormat{ % #1: depth, #2:
  position at that depth
    \int_compare:nNnTF {#1} = {1}
    { \int_to_Roman:n {#2} } %%
      outermost: I, II, III, ...
    { . \int_to_arabic:n {#2} } %%
      deeper: .1, .2, .3, ...
}
\ExplSyntaxOff

\begin{RpgMap}[header-offset=2]
    \area{Gatehouse} The only way in.

    \begin{RpgArea}{Keep}
        \area{Armoury} Racks of old
          spears.

        \begin{RpgArea}{Cellars}
            \area{Wine Store} Mostly
              empty.
        \end{RpgArea}
    \end{RpgArea}
\end{RpgMap}
```

Area I: Gatehouse
The only way in.
Area II: Keep
Area II.1: Armoury. Racks of old spears.
Area II.2: Cellars.
Area II.2.1: Wine Store. Mostly empty.

`\RpgThemeSetFiligree { code }`

Stores the `tikz` code which draws the filigree frame (page 21). It is drawn over the box, with `FiligreeColor` as the line colour, and may refer to the box's corners through the `frame` node (such as `frame.north west`). The size of the design should be scaled by `\RpgFiligreeSize` (page 11).

A Custom Filigree

```
\RpgThemeSetFiligree{
    %% a bracket at each corner, scaled
      by \RpgFiligreeSize
    \foreach \corner/\dx/\dy in
        {north west/1/-1, north
          east/-1/-1, south west/1/1,
          south east/-1/1}
    {
        \draw[thick]
            ($(frame.\corner)+(0,{2*\dy*\RpgFiligreeSize},0)$)
            -- (frame.\corner)
            --
              ($(frame.\corner)+({2*\dx*\RpgFiligreeSize},0)$);
    }
}

\begin{RpgFiligree}
    The frame is drawn over the box, so
      it can extend beyond its edges.
\end{RpgFiligree}
```

The frame is drawn over the box, so it can extend beyond its edges.

`\RpgThemeSetSecretHidden { code }`

Stores the code which typesets an `RpgSecret` while secrets are hidden. The available arguments are:

Argument	Value
`#1`	The public text (possibly empty).
`#2`	The secret text.
`#3`	Any options given to the environment which are not recognised by it, intended to be passed to a `tcolorbox`.

By default, only the public text (`#1`) is typeset.

`\RpgThemeSetSecretVisible { code }`

Stores the code which typesets an `RpgSecret` while secrets are shown. The available arguments are the same as for `\RpgThemeSetSecretHidden`. By default, the secret text is typeset in a `tcolorbox` with the `rpgsecret` style, with the public text (if any) in its lower part.

The following example sets both commands, so that secrets appear inline, and are revealed in brackets after the public text:

Inline Secrets

```
\RpgThemeSetSecretHidden{#1}
\RpgThemeSetSecretVisible{#1
  \textcolor{SecretColor!60!black}{[#2]}}

\RpgSwitch{ShowSecrets}{false}
Players: the door \begin{RpgSecret}{is
  locked}is trapped\end{RpgSecret}.

\RpgSwitch{ShowSecrets}{true}
GM: the door \begin{RpgSecret}{is
  locked}is trapped\end{RpgSecret}.
```

Players: the door is locked.
GM: the door is locked [is trapped].

Table of Contents

Themes lay out the table of contents with the `\titlecontents` command of the `titletoc` package. Chapters in the appendices are usually listed differently from the others: in the default theme, as 'Appendix A: Title', followed by a page number. To allow this, `RPGTeX` provides two values which change when `\appendix` is called, for use in the `chapter` entry. Since they are changed through the `.toc` file, they only change within the table of contents itself.

`\RpgTocChapterName` *(Variable)*

The word placed before the number of a chapter: empty before `\appendix`, and `Appendix` after it.

`\RpgTocChapterPage` *(Variable)*

The filler and page number of a chapter: empty before `\appendix`, and a dotted leader followed by the page number after it.

For example, the default theme lists chapters with:

```
\titlecontents{chapter}[0pt]{\RpgFontTocChapter}
{ \RpgTocChapterName\ \thecontentslabel:~}{}
{\RpgTocChapterPage }
```

Like any change to the formatting of the contents, this should be wrapped in `\RpgIfPreamble` (page 63) (page 61).

Chapter 13: Rule Environments

Rule environments (page 25) are the elements of a document which depend on the rules of a particular game, such as items, spells and monsters. `RPGTeX` itself only creates them: the theme decides which properties each one accepts, and how it is displayed. `RPGTeX` creates four rule environments, `RpgItem`, `RpgSpell`, `RpgFeat` and `RpgStat`, each of which may be displayed either as text or as a card. Designers may also create their own.

Defining a rule environment has three steps:

1. **Create** the environment, with `\RpgThemeNewRuleEnv` or `\RpgThemeNewCardableRuleEnv`. This is only necessary for new environments, since the four standard ones already exist.
2. **Add properties** to it, with the `\Rpg[X]AddProperty` and `\Rpg[X]AddBoolean` commands generated for each environment.
3. **Set its format**, with the `\RpgTheme[X]Format` (and, for a card, `\RpgTheme[X]CardFormat`) command generated for each environment.

Throughout this chapter, `[X]` stands for the name of the environment, so that the commands for `RpgItem` are `\RpgItemAddProperty`, `\RpgThemeItemFormat` and so on.

Creating a Rule Environment

```
\RpgThemeNewCardableRuleEnv{Potion}{UsePotionCard}
\RpgPotionAddProperty{colour}{\PotionColour}{clear}

\RpgThemePotionFormat{
    \paragraph{#1} (\PotionColour) #2
}
\RpgThemePotionCardFormat{
    {\RpgFontCardTitle #1}\par
    \textit{A \PotionColour{} liquid}\par
    #2
}

\begin{RpgPotion}{Healing Draught}[colour=red]
    Restores \RpgDice{2d4+2} hit points.
\end{RpgPotion}

\RpgSwitch{UsePotionCard}{on}
\begin{RpgPotion}[width=4.5cm,
  height=3cm]{Healing Draught}[colour=red]
    Restores \RpgDice{2d4+2} hit points.
\end{RpgPotion}
\RpgSwitch{UsePotionCard}{off}
```

Healing Draught. (red) Restores 2d4+2 hit points.

Healing Draught
A red liquid
Restores 2d4+2 hit points.

Creating Rule Environments

`\RpgThemeNewCardableRuleEnv` `[<prefix>]{ name }{ switch }[<card-options>][<properties>]`

Creates the environment `<prefix><name>` (the prefix is `Rpg` by default), which may be displayed either as text or as a card, together with a new switch (page 17) which chooses between them. The switch is off (text mode) by default. The environment is used as:

```
\begin{<prefix><name>}[<card-options>]{<title>}[<properties>]
  <body>
\end{<prefix><name>}
```

In card mode, the `card-options` are passed to `RpgCard` (page 24); in text mode they are ignored. The optional `card-options` and `properties` given to `\RpgThemeNewCardableRuleEnv` are the defaults used when the corresponding argument is omitted. They are replaced, rather than combined, when the argument is given.

In addition to the commands generated by `\RpgThemeNewRuleEnv`, this creates `\RpgTheme[X]CardFormat`.

Creating an environment which already exists redefines it, resetting its formats to their defaults; an existing switch keeps its current value.

`\RpgThemeNewRuleEnv [<prefix>]{ name }`

Creates the environment `<prefix><name>` (the prefix is `Rpg` by default), which is always displayed as text. The environment is used as:

```
\begin{<prefix><name>}{<title>}[<properties>]
  <body>
\end{<prefix><name>}
```

It also creates the commands `\Rpg[X]AddProperty`, `\Rpg[X]AddBoolean` and `\RpgTheme[X]Format`, using the same prefix.

Creating an environment which already exists redefines it, resetting its format to the default.

Properties

Each property is a key which may be given in the `properties` argument of the environment. Its value is stored in a variable, which the format can then use. Properties are set separately for each use of the environment, so a property which is not given takes its default value.

Properties belong to the environment, not to the theme which added them. Adding a property with the same name again replaces it, but a property added by one theme remains available after another theme is loaded.

`\Rpg[X]AddBoolean (*){ key }[<aliases>]{ variable }`

Adds a flag property, which takes no value. The unstarred form sets the boolean `variable` to false, and makes it true when the flag is given; the starred form does the opposite. The optional `aliases` are a comma-separated list of alternative names for the same flag.

`\Rpg[X]AddProperty { key }[<aliases>]{ variable }{ default }`

Adds a property which takes a value, stored in `variable`, with the given `default`. The optional `aliases` are a comma-separated list of alternative names for the same property.

Within either format, the value of a property may also be retrieved by its name with `\RpgGetKey` (page 26). This does not work for flag properties, whose variables should be tested directly (for example, with `\bool_if:NTF`).

Formats

The format of a rule environment is a code setter (see the sidebar (page 61)). Both formats have the same arguments:

Argument	Value
`#1`	The title of the element.
`#2`	The body of the environment.

The values of the properties are available through their variables (or `\RpgGetKey`). The following examples restyle the `RpgPotion` environment created at the start of this chapter; a format set within an example applies only to that example.

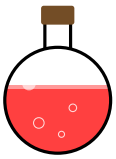
`\RpgTheme[X]CardFormat { code }`

Stores the code which typesets the element in card mode. It is typeset inside an `RpgCard` (page 24), and so continues onto further cards if it is too long for one. By default, the title is typeset as a `\subsection`, followed by the body.

A Custom Card Format

```
\RpgPotionAddProperty{colour}{\PotionColour}{gray}
\RpgPotionAddProperty{rarity}{\PotionRarity}{common}

\RpgThemePotionCardFormat{
    \begin{center}
        {\RpgFontCardTitle #1}\par
        \textcolor{\PotionColour!70!black}{\rule{0.5\linewidth}{1pt}}\par\medskip
        \vfil

        \begin{tikzpicture}
            %% the liquid, filling the lower
              part of the flask
            \begin{scope}
                \clip (0,0) circle (0.7);
                \fill[\PotionColour!75]
                  (-0.7,-0.7) rectangle
                  (0.7,0.2);
                \fill[\PotionColour!40]
                  (-0.7,0.15) rectangle
                  (0.7,0.2);
                \foreach \x/\y/\r in
                  {-0.25/-0.3/0.07,
                  0.2/-0.1/0.05, 0.05/-0.45/0.04}
                    { \draw[white, opacity=0.8]
                      (\x,\y) circle (\r); }
            \end{scope}
            %% the flask, its neck and the cork
            \draw[very thick] (-0.17,0.68) --
              (-0.17,1.05) (0.17,0.68) --
              (0.17,1.05);
            \draw[very thick] (0,0) circle (0.7);
            \fill[brown!60!black, rounded
              corners=1pt] (-0.22,1.0) rectangle
              (0.22,1.25);
            \fill[white, opacity=0.6]
              (-0.38,0.22) circle (0.09);
        \end{tikzpicture}\par\medskip
        \textsc{\PotionRarity}\par\medskip
        \vfil

        \small\itshape #2

        \vfil
    \end{center}
}

\RpgSwitch{UsePotionCard}{on}
\begin{RpgPotion}[width=5cm, height=6cm]{Potion
  of Healing}[colour=red, rarity=uncommon]
    Restores \RpgDice{2d4+2} hit points when
      drunk.
\end{RpgPotion}
\begin{RpgPotion}{Flask of Oil}[colour=orange]
    When thrown or broken, covers all creatures
      and objects within 5ft in oil. The next
      time a creature covered in oil takes fire
      damage, they take \RpgDice{3d8}
      additional fire damage.
\end{RpgPotion}
\RpgSwitch{UsePotionCard}{off}
```

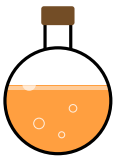
Potion of Healing



UNCOMMON

Restores 2d4+2 hit points when drunk.

Flask of Oil



COMMON

When thrown or broken, covers all creatures and objects within 5ft in oil. The next time a creature covered in oil takes fire damage, they take 3d8 additional fire damage.

`\RpgTheme[X]Format { code }`

Stores the code which typesets the element in text mode. By default, the title is typeset as a `\subsection`, followed by the body.

A Custom Text Format

```
\RpgPotionAddProperty{colour}{\PotionColour}{gray}
\RpgPotionAddProperty{rarity}{\PotionRarity}{common}
\RpgPotionAddBoolean{magic}{\PotionMagic}

\ExplSyntaxOn
\RpgThemePotionFormat{ % #1: title, #2: body
   \begin{tcolorbox}[
          enhanced, sharp~corners,
            boxrule=0pt, leftrule=4pt,
          colframe=\PotionColour!70!black,
            colback=\PotionColour!7,
          colbacktitle=\PotionColour!70!black,
            coltitle=white,
          title={
              \textbf{#1}
              \bool_if:NT \PotionMagic
              {
                  \hspace{0.4em}
                    \tikz[baseline=-0.6ex]
                    \fill[yellow!80!orange]
                      (0,4pt) -- (1pt,1pt) --
                        (4pt,0) -- (1pt,-1pt)
                      -- (0,-4pt) --
                        (-1pt,-1pt) -- (-4pt,0)
                        -- (-1pt,1pt) -- cycle;
              }
              \hfill \textit{\PotionRarity}
          },
       ]
       #2
   \end{tcolorbox}
}
\ExplSyntaxOff

\begin{RpgPotion}{Potion of
  Healing}[colour=red, rarity=uncommon, magic]
    Restores \RpgDice{2d4+2} hit points when
      drunk.
\end{RpgPotion}
\begin{RpgPotion}{Flask of Oil}[colour=orange]
    When thrown or broken, covers all creatures
      and objects within 5ft in oil. The next
      time a creature covered in oil takes fire
      damage, they take \RpgDice{3d8}
      additional fire damage.
\end{RpgPotion}
```

Potion of Healing ✦ *uncommon*

Restores 2d4+2 hit points when drunk.

Flask of Oil *common*

When thrown or broken, covers all creatures and objects within 5ft in oil. The next time a creature covered in oil takes fire damage, they take 3d8 additional fire damage.

PART IV

Appendices

Appendix A: AI Usage

This project began as a human-led passion project. For the longest time the authors refused to utilise automated coding tools, but eventually it was decided that this was a bit of a Luddite opinion. An AI assistant (Claude Code) was only permitted to alter the code near the very end, and only at the explicit direction of the authors.

We believe it is important to disclose not only where AI was used, but where *AI did things differently than the human authors would have.*

Package Code

The following changes encompass all of the changes made by the Claude Code tool. Of these, we consider the vast majority to have been time-saving changes that the human authors would have done identically:

1. Some minor bugfixes
   - Split the logic of the `\RpgWholePageImage` into a container function to allow a `\label` to attach properly to a `\part` and reference the correct page
   - Reset the styling of `\thepart` when a theme is reset, to prevent changes persisting between themes
   - Fixed a bug with the colour assignments in the dnd-theme `RpgStat` block.
   - Fixed some platform-specific bugs in the `rpglatex` compiler tool
2. Some renaming of variables to fit the established style guide and fix typos
3. Replace theme `\NewDocument[X]` with `\DeclareDocument[X]` to allow themes to be repeatedly set, and equivalent changes for expl3 declarations.
4. Removed dead code left over from previous drafts

There are only two places in this package where the tool made changes that differ from what the human authors would have written:

1. The updated auto-configuration used to populate `\RpgPackagePath` replaced the human-written version which relied on a fragile call to `\write18`. The AI suggested relying on `\sys_get_shell`, which we did not know permitted limited calls to `kpsewhich`. This provided a vastly simpler interface for the majority of users.
2. The filigree applied in the SciFi theme was generated entirely by the AI without human description. This was mainly done out of an interest to see what the tool would generate. We like the result, so kept it in.

Documentation

We allowed the AI to be significantly more involved in the writing of the documentation. Even then, in most cases this involved updating a previous version of the documentation to incorporate the new names and syntax after a rewrite of the core engine.

The `scifi` theme chapter, and the `dnd` statblock section are exceptions: here we allowed the tool to generate a first draft, and then revised it by hand.

Appendix B: The rpglatex Compiler

rpglatex is a Python 3 script (scripts/rpglatex) which wraps xelatex or lualatex, adding several conveniences for compiling RPGTeX documents. It is supported on Linux and macOS only; on Windows, documents should be compiled with xelatex or lualatex directly (page 1). To use it from any directory, place it on your PATH, for example with a symbolic link:

bash

```
ln -s ~/path/to/rpgtex/scripts/rpglatex ~/.local/bin/rpglatex
```

rpglatex [options] <file>

Compiles <file>, performing the following steps automatically:

- Defines \RpgPackagePath, so that RPGTeX does not need to locate itself.
- Runs the compiler twice, so that references and tikz remember picture commands resolve correctly (three times with -full).
- Runs bibtex between the passes, if requested with -ref, and makeindex whenever the document produces an index.
- Keeps the auxiliary files (.aux, .log and so on) in a separate build directory, and moves only the finished PDF to the output location.
- Hides the compiler's lengthy output, printing a short summary instead. If compilation fails, the compiler's output is printed so that the error can be found.

The following options are available:

Option	Effect	Default
-b, -build <dir>	The directory in which auxiliary files are stored.	.build
-f, -full	Runs the compiler three times, rather than twice, for very complex or reference-heavy documents.	off
-l, -lualatex	Compiles with lualatex.	off
-o, -output <name>	The name of the finished PDF.	<file>.pdf
-p, -print	Compiles in print mode, by setting bg=print through the command line interface (page 11).	off
-q, -quiet	Suppresses the script's own progress messages.	off
-r, -ref	Runs bibtex after the first pass.	off
-show <0/1>	Whether to open the finished PDF (or switch to it, if already open) in a viewer.	1
-v, -verbose	Shows the compiler's full output.	off
-x, -xelatex	Compiles with xelatex.	on

For example, the following compiles a print version of adventure.tex with lualatex, without opening it:

```
rpglatex adventure.tex -l -p -show 0
```

Appendix C: Dependencies

RPGTeX loads the packages listed below. All of them are included in a full TeX Live installation, and MiKTeX installs missing packages automatically the first time they are needed. On a minimal installation, missing packages may be added with tlmgr:

```bash
tlmgr install <package>
```

Some files are distributed as part of a larger package, whose name must be given to tlmgr instead; this name is shown in brackets. Packages marked with an asterisk are only loaded in some circumstances, which are described beneath the list.

adjustbox
array (tools)
contour
cuted (sttools)
enumitem
eso-pic
extarticle* (extsizes)
extbook* (extsizes)
fancyhdr
float
fontenc* (latex)
fontspec*
geometry*
graphicx (graphics)
hang
l3keys2e (l3packages)
lettrine
luacolor*
magaz
microtype
multitoc* (ms)
nameref (hyperref)
ragged2e
relsize
Royal (cfr-initials)
standalone*
stfloats (sttools)
supertabular
tabularx (tools)
tcolorbox
tikz (pgf)
titlesec
titletoc (titlesec)
tocloft
xcolor
xltabular
xparse (l3packages)

The packages marked with an asterisk are loaded as follows:

- extarticle and extbook: as the base classes of rpghandout and rpgbook respectively.
- fontenc and fontspec: when the font option is active (page 8).
- geometry: by the RPGTeX classes.
- luacolor: when compiling with lualatex.
- multitoc: by the RPGTeX classes, unless the nomultitoc option is used.
- standalone: by the rpgcard and rpgdeck classes.

The rpgcard and rpgdeck classes are built on the standard article class, which is part of every LaTeX installation, as is expl3.

Fonts

The default and dnd themes use fonts from the TeX distribution, while the fonts of the scifi theme are distributed with RPGTeX itself, in themes/scifi/fonts. Fonts are only loaded when the font option is active. The default theme uses Latin Modern, which is part of every TeX distribution; the dnd theme uses:

- texgyrebonum-*.otf (tex-gyre): TeX Gyre Bonum, the main body font.
- EBGaramond-Regular.otf (ebgaramond): EB Garamond, used for small capitals.
- KpSans-*.otf and KpRoman-*.otf (kpfonts-otf): Kp-Fonts, used for titles and table headings.
- GilliusADFNo2-*.otf (gillius): Gillius ADF No2, used for subtitles and table text.

Appendix D: Image Credits

The following images are used in this document. We are grateful to their creators, and to the organisations which make them freely available.

Typewriter Used on the cover and on the title page of the Appendices.

Keyboard, Typewriter, Antiquity by `bjoern-hiekel`, from Pixabay. Used under the Pixabay Content License.

The Writing Desk Used on the title page of Part I.

Childe Hassam, *The Writing Desk* (1915), etching in black on wove paper. National Gallery of Art, Washington, Corcoran Collection (Bequest of Frank B. Bristow, 1968), accession no. 2015.19.1530. Available through the National Gallery of Art's Open Access programme, under a CC0 licence.

Costume Used on the title page of Part II.

Mary E. Humes, *Costume* (c. 1941), watercolour, graphite, and pen and ink on paper. National Gallery of Art, Washington, Index of American Design, accession no. 1943.8.9509. Available through the National Gallery of Art's Open Access programme, under a CC0 licence.

Desk Used on the title page of Part III.

Ferdinand Cartier, *Desk* (c. 1940), graphite on paperboard. National Gallery of Art, Washington, Index of American Design, accession no. 1943.8.5338. Available through the National Gallery of Art's Open Access programme, under a CC0 licence.

Enrag'd Monster Used on the example cover pages in the theme chapters.

John Hamilton Mortimer, *Enrag'd Monster* (1778), etching. National Gallery of Art, Washington, Ailsa Mellon Bruce Fund, accession no. 1974.53.22. Available through the National Gallery of Art's Open Access programme, under a CC0 licence.

The Mermaid Used on the example part pages in the theme chapters.

Frederick Stuart Church, *The Mermaid* (c. 1881), etching in green ink. National Gallery of Art, Washington, Reba and Dave Williams Collection, Gift of Reba and Dave Williams, accession no. 2008.115.1238. Available through the National Gallery of Art's Open Access programme, under a CC0 licence.

Paper Texture Used for the pages of the chapter on the `dnd` theme, and available to documents through `\RpgSetDndPaper` (page 36).

From Lost and Taken, by way of the D&D 5e LaTeX Template, from which `RPGTeX` is forked. Lost and Taken's terms allowed its textures to be used royalty-free, with no attribution required.

The Crown Jewels Used for the example items in the chapters on the default and `dnd` themes.

Blaise Alexandre Desgoffe, *Les Joyaux de la Couronne* (*The Crown Jewels*; 19th century), painting. From Wikimedia Commons, where it is in the public domain, as a faithful reproduction of a public domain work of art.

Phaser Model Used for the example item in the chapter on the `scifi` theme.

Marcin Wichary, *star trek phaser model* (2008), photograph, from Wikimedia Commons (originally published on Flickr). Used under the CC BY 2.0 licence; the image has been reduced in size.

Index

abstract, 21
action, 45
appendix, 74
area, 23, 24
attack, 45, 46
atWill, 49
author, 12, 13

background (flag), 8
bg (key), 8

cardbreak, 24
chapter, 14, 67
chapter*, 14
chaptermark, 12, 14
columns (key), 8
cover, 12, 13, 30, 70

emph, 9, 16, 67
eso-pic, 17

font (flag), 8
footnote, 24, 40

galaxy, 54
gillius, 33
gyreBonum, 33

jura, 54
justified (flag), 9

kepler, 33
keplerserif, 33

level, 49
lrfont, 28

maketitle, 6, 7, 9, 12, 13, 29, 67, 69, 70
melee, 46

name, 44
noemph (flag), 9
nomultitoc (flag), 9
normalfont, 28, 33, 54, 67

oneside (flag), 9
openright (flag), 9

paragraph, 68
part, 12-14, 30, 61, 67, 69, 70
part*, 12, 14
perDay, 49

ranged, 46
reaction, 45
Rpg[X], 26
    AddBoolean, 75, 76
    AddProperty, 75, 76
RpgAbstract, 21, 68
RpgAdvanceColors, 35
RpgArea, 24
RpgCantripScaling, 41
RpgCard, 24, 26, 68, 71, 75, 77
    Reset, 25
    Set, 25
RpgChapterName, 14
RpgCMD, 11
RpgContour, 10, 15, 37, 38, 57, 62
RpgContourIfDifferent, 15
RpgCoverTransparency, 13, 17, 30
RpgDefaultPartImage, 13, 30
RpgDice, 15, 31, 38, 43, 58, 71
RpgDropCap, 11, 16, 68, 72
RpgDropCapSeparation, 10, 16
RpgFake, 14
RpgFeat, 26
RpgFiligree, 10, 20, 21
RpgFiligreeSize, 11, 29, 73
RpgFont
    AbstractBody, 21, 68
    AbstractTitle, 21, 68
    Body, 28, 33, 54, 67
    CardBody, 25, 31, 68
    CardHeader, 31, 32, 68
    CardTitle, 31, 68
    Chapter, 67
    DropCap, 16, 68
    DropCapAfter, 16, 68
    Emphasis, 16, 67
    Footer, 29, 36, 56, 68
    Narration, 19, 68
    PageNumber, 29, 36, 56, 68, 69
    Paragraph, 68
    Part, 30, 67, 70
    SecretTitle, 22, 68
    Section, 66, 67
    SidebarBody, 19, 68
    SidebarTitle, 19, 68
    StatBlockBody, 68
    StatBlockSection, 45, 68
    StatBlockTitle, 68
    Subparagraph, 68
    Subsection, 67
    Subsubsection, 67
    SubTitle, 13, 30, 67
    TableBody, 20, 68
    TableHeader, 20, 68
    TableTitle, 20, 68
    TipBody, 19, 68
    TipTitle, 19, 68
    Title, 13, 29, 67
    TocChapter, 68
    TocPart, 68
    TocSection, 68
RpgGetAuthor, 13, 67, 69
RpgGetAuthor*, 13
RpgGetKey, 26, 76
RpgGetName, 14
RpgGetSubtitle, 13, 67, 69
RpgGetSubtitle*, 13
RpgGetTitle, 13, 67, 69
RpgGetTitle*, 13
RpgIf
    ColoursEqual, 62
    ColoursEqualTF, 57, 62
    Env, 26
    Font, 62, 63
    FontTF, 62, 63
    Paper, 63
    PaperTF, 63
    Preamble, 61-63, 74
    PreambleTF, 61, 63
    Sections, 63
    SectionsTF, 63
    Switch, 18
    Title, 63
    TitleTF, 63
    TwoSide, 63
    TwoSideTF, 63
RpgItem, 26
RpgKey, 9, 16, 67
rpglatex, 81
RpgMakeTitle, 12, 13
RpgMap, 23, 72
    Ref, 23, 24
    Ref*, 23
RpgNarration, 10, 19, 50, 68, 71
RpgNewSwitch, 18
RpgOrdinal, 16, 17, 48, 49
RpgPackagePath, 2, 9, 62, 80, 81
RpgPage, 16, 24
RpgPart
    Name, 13, 14
    Transparency, 13, 30
RpgPlural, 17
RpgPut, 17, 69
RpgRowStyle, 20
RpgSecret, 10, 22, 68, 71
RpgSectionName, 14
RpgSetDMGColors, 35
RpgSetDndPaper, 36, 83
RpgSetPHBColors, 35
RpgSetTheme, 7, 9, 12, 52, 61, 62
RpgSetThemeColor, 10, 12
RpgSetThemePath, 9, 12, 61
RpgSidebar, 10, 19, 68, 71
RpgSpell, 26
RpgSpellUpcast, 41
RpgStat, 26, 42, 47, 48, 68, 80
    *, 51
    Action, 45
    Attack, 45
    LegacyMode, 51, 52
    LegendaryDefiance, 50
    LegendaryResistance, 50
    LegendarySection, 49, 50
    Line, 52
    Mod, 44, 46
    MythicSection, 50, 53
    Name, 43, 44, 53
    Reaction, 45
    SaveEffect, 47
    Section, 45
    SetName, 44
    SpellList, 48, 49
    Spells, 48, 49
    SpellSlots, 48, 49
rpgstring
    legendaryDefiance, 53
    legendaryResistance, 53
    legendarySpiel, 53
    mythicDefaultBody, 53
    mythicGains, 53
    mythicNarrationLeadIn, 53
RpgSubsectionName, 14
RpgSubsubsectionName, 14
RpgSuperOrdinal, 17
RpgSwitch, 6, 7, 18, 22, 31, 32, 38-40, 51
RpgTable, 10, 20, 68
RpgTheme
    [X]CardFormat, 75-77
    [X]Format, 75, 76, 78
    BoolKey, 64
    CodeMapKey, 64
    CreateFont, 66
    DiceFormat, 71
    DropCapHeight, 72
    Font, 52, 62, 66, 67
    FooterDecoration, 8, 69
    Keys, 64, 65
    MapFormat, 72
    NewBoolKey, 63
    NewCardableRuleEnv, 75
    NewCodeMapKey, 63, 65
    NewRuleEnv, 75, 76
    NewTLChoiceKey, 64

NewTLKey, 64
NewTLMapKey, 64
OnPartChange, 70
PageNumberAt, 69
PaperColor, 8, 69
PaperImage, 8, 69
PartPage, 61, 70
PartPageTikz, 52
SetColorCycle, 36
SetDMGParts, 37, 52
SetFiligree, 73
SetPHBParts, 37, 52
SetSecretHidden, 73, 74
SetSecretVisible, 74
TitleCover, 70
TitleHeader, 70
TitleMode, 13, 29, 70
TLChoiceKey, 64
TLKey, 64
TLMapKey, 64
RpgTip, 10, 19, 21, 68, 71
RpgTocChapterName, 74
RpgTocChapterPage, 74
RpgToggle, 18
RpgWholePageImage, 17

save, 47
scifi, 54, 57
section, 14, 45, 66, 67
section*, 14, 23
sectionmark, 12, 14
sections (flag), 9
size (key), 9
spells, 48
stat, 44
subparagraph, 68
subparagraph*, 23
subsection, 14, 31, 32, 39, 67, 77, 78
subsection*, 14, 23
subsectionmark, 12, 14
subsubsection, 14, 38, 40, 67
subsubsection*, 14
subsubsectionmark, 12, 14
subtitle, 13
summary, 21

theme (key), 9
themepath (key), 9
thepart, 70
title, 12, 13
title (flag), 9